LAPORAN PRAKTIKUM
PEMROGRAMAN MOBILE
MODUL 3

BUILD A SCROLLABLE LIST

Oleh:

Hafiz Perdana NIM. 2410817210027

PROGRAM STUDI TEKNOLOGI INFORMASI
FAKULTAS TEKNIK
UNIVERSITAS LAMBUNG MANGKURAT
APRIL 2026

LEMBAR PENGESAHAN

LAPORAN PRAKTIKUM PEMROGRAMAN MOBILE

MODUL 3

Laporan Praktikum Pemrograman Mobile Modul 3: Build a Scrollable List ini disusun sebagai syarat lulus mata kuliah Praktikum Pemrograman Mobile. Laporan Praktikum ini dikerjakan oleh:

Nama Praktikan : Hafiz Perdana
NIM : 2410817210027

Menyetujui, Asisten Praktikum	Mengetahui, Dosen Penanggung Jawab Praktikum
Muhammad Erza Raihan NIM. 2310817210027	Irham Maulani Abdul Gani, S.Kom., M.Kom NIP. 199710312025061009

DAFTAR ISI

LEMBAR PENGESAHAN ii
DAFTAR ISI iii
DAFTAR GAMBAR iv
DAFTAR TABEL v
SOAL 1 6
A. Source Code 9
B. Output Program 43
C. Pembahasan 45
SOAL 2 66
A. Pembahaan 66
SOAL 3 67
A. Pembahasan 67
ADDITIONAL 68
A. Pembahasan 68
TAUTAN GIT 69

DAFTAR GAMBAR

Gambar 1. Contoh UI List 7
Gambar 2. Contoh UI Detail 7
Gambar 3. Contoh Penerapan LazyRow/Carousel 8
Gambar 4. Screenshot Hasil Jawaban Soal 1 Compose 43
Gambar 5. Screenshot Hasil Jawaban Soal 1 XML 44

DAFTAR TABEL

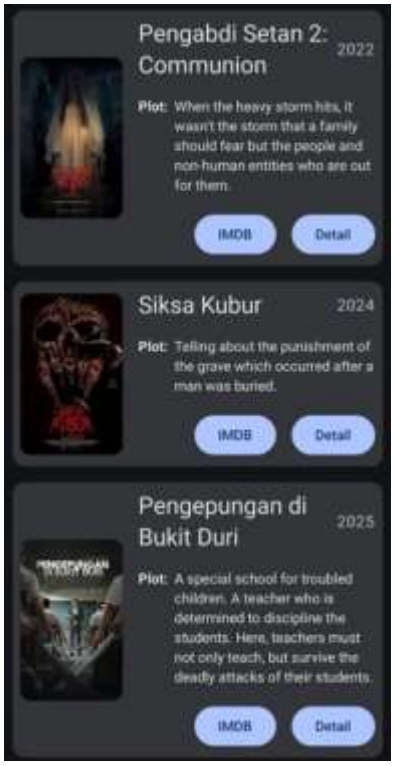
Tabel 1. Source Code GameData.kt Compose Soal 1 9
Tabel 2. Source Code Game.kt Compose Soal 1 10
Tabel 3. Source Code AppNavigation.kt Compose Soal 1 10
Tabel 4. Source Code DetailScreen.kt Compose Soal 1 11
Tabel 5. Source Code HomeScreen.kt Compose Soal 1 13
Tabel 6. Source Code SettingsScreen.kt Compose Soal 1 16
Tabel 7. Source Code Theme.kt Compose Soal 1 18
Tabel 8. Source Code MainActivity.kt Compose Soal 1 19
Tabel 9. Source Code Game.kt XML Soal 1 20
Tabel 10. Source Code GameData.kt XML Soal 1 20
Tabel 11. Source Code DetailFragment.kt XML Soal 1 22
Tabel 12. Source Code GameListAdapter.kt XML Soal 1 24
Tabel 13. Source Code HighlightAdapter.kt XML Soal 1 25
Tabel 14. Source Code HomeFragment.kt XML Soal 1 26
Tabel 15. Source Code SettingsFragment.kt XML Soal 1 29
Tabel 16. Source Code LanguageManager.kt XML Soal 1 30
Tabel 17. Source Code MainActivity.kt XML Soal 1 32
Tabel 18. Source Code activity_main.xml XML Soal 1 32
Tabel 19. Source Code fragment_detail.xml XML Soal 1 33
Tabel 20. Source Code fragment_home.xml XML Soal 1 35
Tabel 21. Source Code fragment_settings.xml XML Soal 1 36
Tabel 22. Source Code item_game.xml XML Soal 1 37
Tabel 23. Source Code item_hightlight_game.xml XML Soal 1 40
Tabel 24. Source Code menu_home.xml XML Soal 1 41
Tabel 25. Source Code nav_graph.xml XML Soal 1 41

SOAL 1

Buatlah sebuah aplikasi Android menggunakan XML dan Jetpack Compose yang dapat menampilkan list dengan ketentuan berikut:

1. List menggunakan fungsi RecyclerView (XML) dan LazyColumn (Compose)
2. List paling sedikit menampilkan 5 item. Tema item yang ingin ditampilkan bebas
3. Item pada list menampilkan teks dan gambar sesuai dengan contoh di bawah
4. Terdapat 2 button dalam list, dengan fungsi berikut:
    a. Button pertama menggunakan intent eksplisit untuk membuka aplikasi atau browser lain
    b. Button kedua menggunakan Navigation component untuk membuka laman detail item
5. Sudut item pada list dan gambar di dalam list melengkung atau rounded corner menggunakan Radius
6. Saat orientasi perangkat berubah/dirotasi, baik ke portrait maupun landscape, aplikasi responsif dan dapat menunjukkan list dengan baik. Data di dalam list tidak boleh hilang
7. Aplikasi menggunakan arsitektur single activity (satu activity memiliki beberapa fragment)
8. Aplikasi berbasis XML harus menggunakan ViewBinding
9. Aplikasi Mendukung Multi-language (bahasa Indonesia dan Inggris)
10. Terdapat 1 halaman/screen untuk melakukan pergantian bahasa
11. Gunakan Icon dari library ‘material’ (material,material2,material3) untuk membuat tombol yang mengarahkan ke halaman untuk melakukan pergantian bahasa, dan letakkan di bagian header/heading homescreen utama
12. Diatas Scrollable list, terdapat sebuah “Item Highlight” yang menggunakan Carousel atau LazyRow dengan ketentuan:
    - Item bisa bergeser secara finite atau infinite loop sebanyak jumlah item yang ada pada scrollable-list

UI item list harus berisi 1 gambar, 2 button (intent eksplisit dan navigasi), dan 2 baris teks dan setiap baris memiliki 2 teks yang berbeda. Diusahakan agar desain UI item list menyerupai UI berikut:



Gambar 1. Contoh UI List

Desain UI laman detail bebas, tetapi diusahakan untuk mengikuti kaidah desain Material Design dan data item ditampilkan penuh di laman detail seperti contoh berikut:

Gambar 2. Contoh UI Detail

Gambar 3. Contoh Penerapan LazyRow/Carousel

Item yang bergeser bisa secara finite (mentok kiri kanan) atau secara infinite loop (misal dari item terakhir ‘5’kembali ke item awal ‘1’)

A. Source Code

COMPOSE

Package data - GameData.kt

Tabel 1. Source Code GameData.kt Compose Soal 1

```
package com.example.modul3compose.data

import com.example.modul3compose.R
import com.example.modul3compose.model.Game

object GameData {
    val gameList = listOf(
        Game(
            1,
            “Honkai: Star Rail”,
            “2022”,
            “Perjalanan melintasi bintang-bintang di atas Astral Express.”,
            “A journey across the stars aboard the Astral Express.”,
            R.drawable.img_hsr,
            “https://hsr.hoyoverse.com/”),
        Game(
            2,
            “Arknights: Endfield”,
            “2026”,
            “Eksplorasi planet Talos-II yang berbahaya dan penuh misteri.”,
            “Explore the dangerous and mysterious planet of Talos-II.”,
            R.drawable.img_endfield,
            “https://endfield.gryphline.com/en-us”),
        Game(
            3,
            “Clair Obscur: Expedition 33”,
            “2025”,
            “Misi terakhir untuk menghancurkan entitas yang menghapus umat manusia.”,
            “The final mission to destroy an entity erasing humanity.”,
            R.drawable.img_coe33,
“https://store.steampowered.com/app/1903340/Clair_Obscur_Expedition_33/”),
        Game(
            4,
```

```
            “Persona 5 Royal”,
            “2019”,
            “Kisah pencuri hati (Phantom Thieves) yang mengubah masyarakat.”,
            “The story of Phantom Thieves changing corrupt society.”,
            R.drawable.img_p5r,
“https://store.steampowered.com/app/1687950/Persona_5_Royal/”),
        Game(
            5,
            “Persona 3 Reload”,
            “2024”,
            “Mengungkap misteri Dark Hour dan menyelamatkan dunia.”,
            “Uncover the mystery of the Dark Hour and save the world.”,
            R.drawable.img_p3r,
“https://store.steampowered.com/app/2161700/Persona_3_Reload/”)
    )
}
```

Package model - Game.kt

Tabel 2. Source Code Game.kt Compose Soal 1

```
package com.example.modul3compose.model

data class Game(
    val id: Int,
    val title: String,
    val year: String,
    val plotId: String,
    val plotEn: String,
    val imageRes: Int,
    val url: String
)
```

Package ui => navigation - AppNavigation.kt

Tabel 3. Source Code AppNavigation.kt Compose Soal 1

```
package com.example.modul3compose.ui.navigation

import androidx.compose.runtime.Composable
```

```
4  import androidx.navigation.compose.NavHost
5  import androidx.navigation.compose.composable
6  import androidx.navigation.compose.rememberNavController
7  import com.example.modul3compose.ui.screen.DetailScreen
8  import com.example.modul3compose.ui.screen.HomeScreen
9  import com.example.modul3compose.ui.screen.SettingsScreen
10
11 @Composable
12 fun AppNavigation() {
13     val navController = rememberNavController()
14
15     NavHost(navController = navController, startDestination
   = "home") {
16         composable("home") {
17             HomeScreen(navController = navController)
18         }
19         composable("detail/{gameId}") { backStackEntry ->
20             val gameId =
   backStackEntry.arguments?.getString("gameId")?.toIntOrNull()
21             DetailScreen(navController = navController,
   gameId = gameId)
22         }
23         composable("settings") {
24             SettingsScreen(navController = navController)
25         }
26     }
27 }
```

Package ui => screen - DetailScreen.kt

Tabel 4. Source Code DetailScreen.kt Compose Soal 1

```
1  package com.example.modul3compose.ui.screen
2
3  import androidx.compose.foundation.*
4  import androidx.compose.foundation.layout.*
5  import androidx.compose.foundation.shape.RoundedCornerShape
6  import androidx.compose.material.icons.Icons
7  import
   androidx.compose.material.icons.automirrored.filled.ArrowBa
   ck
8  import androidx.compose.material3.*
9  import androidx.compose.runtime.Composable
10 import androidx.compose.ui.*
11 import androidx.compose.ui.draw.clip
12 import androidx.compose.ui.graphics.Color
13 import androidx.compose.ui.layout.ContentScale
```

```
import androidx.compose.ui.platform.LocalConfiguration
import androidx.compose.ui.res.painterResource
import androidx.compose.ui.text.font.FontWeight
import androidx.compose.ui.unit.dp
import com.example.modul3compose.data.GameData

@Composable
fun DetailScreen(navController: androidx.navigation.NavController, gameId: Int?) {
    val game = GameData.gameList.find { it.id == gameId }
    val lang = LocalConfiguration.current.locales.get(0).language
    val isId = lang == "in" || lang == "id"

    Scaffold(
        topBar = {
            Surface(color = Color.Black, shadowElevation = 4.dp) {
                Row(Modifier.fillMaxWidth().statusBarsPadding().height(64.dp).padding(horizontal = 8.dp), verticalAlignment = Alignment.CenterVertically) {
                    IconButton(onClick = { navController.popBackStack() }) {
                        Icon(Icons.AutoMirrored.Filled.ArrowBack, null, tint = Color.White)
                    }
                    Text(game?.title ?: "Detail", style = MaterialTheme.typography.titleLarge, fontWeight = FontWeight.Bold, modifier = Modifier.padding(start = 8.dp))
                }
            }
        }
    ) { p ->
        game?.let {
            Column(Modifier.fillMaxSize().padding(p).verticalScroll(rememberScrollState()).padding(16.dp)) {
                Image(
                    painterResource(it.imageRes), null,
                    modifier = Modifier.fillMaxWidth().height(260.dp).clip(RoundedCornerShape(16.dp)),
                    contentScale = ContentScale.Crop
                )

```

```

Row(Modifier.fillMaxWidth().padding(vertical = 20.dp), horizontalArrangement = Arrangement.SpaceBetween) {
                    Text(it.title, style = MaterialTheme.typography.headlineMedium, fontWeight = FontWeight.Bold, modifier = Modifier.weight(1f))
                    Text(it.year, style = MaterialTheme.typography.titleLarge, color = Color.Gray)
                }

                Text("Plot", fontWeight = FontWeight.Bold)
                Text(if (isId) it.plotId else it.plotEn, modifier = Modifier.padding(top = 8.dp))
            }
        }
    }
}
```

Package ui => screen - HomeScreen.kt

Tabel 5. Source Code HomeScreen.kt Compose Soal 1

```
package com.example.modul3compose.ui.screen

import android.content.Intent
import androidx.compose.foundation.Image
import androidx.compose.foundation.layout.*
import androidx.compose.foundation.lazy.*
import androidx.compose.foundation.shape.RoundedCornerShape
import androidx.compose.material.icons.Icons
import androidx.compose.material.icons.filled.Settings
import androidx.compose.material3.*
import androidx.compose.runtime.Composable
import androidx.compose.ui.*
import androidx.compose.ui.draw.clip
import androidx.compose.ui.graphics.Color
import androidx.compose.ui.layout.ContentScale
import androidx.compose.ui.platform.*
import androidx.compose.ui.res.*
import androidx.compose.ui.text.font.FontWeight
import androidx.compose.ui.text.style.TextOverflow
import androidx.compose.ui.unit.*
import androidx.core.net.toUri
import com.example.modul3compose.R
import com.example.modul3compose.data.GameData
import com.example.modul3compose.model.Game

```

```
26 @Composable
27 fun HomeScreen(navController:
   androidx.navigation.NavController) {
28     Scaffold(
29         topBar = {
30             Surface(color = Color.Black, shadowElevation =
   4.dp) {
31                 Row(
32                     modifier =
   Modifier.fillMaxWidth().statusBarsPadding().height(64.dp).p
   adding(horizontal = 16.dp),
33                     verticalAlignment =
   Alignment.CenterVertically
34                 ) {
35                     Text(stringResource(R.string.app_name),
   style = MaterialTheme.typography.titleLarge, fontWeight =
   FontWeight.Bold, modifier = Modifier.weight(1f))
36                     IconButton(onClick = {
   navController.navigate("settings") }) {
37                         Icon(Icons.Default.Settings, null,
   tint = Color.White)
38                     }
39                 }
40             }
41         }
42     ) { p ->
43         LazyColumn(modifier =
   Modifier.fillMaxSize().padding(p)) {
44             item {
45
   Text(stringResource(R.string.title_highlight), modifier =
   Modifier.padding(16.dp), fontWeight = FontWeight.SemiBold)
46                 LazyRow(contentPadding =
   PaddingValues(horizontal = 16.dp), horizontalArrangement =
   Arrangement.spacedBy(12.dp)) {
47                     items(GameData.gameList) { game ->
48                         Card(
49                             onClick = {
   navController.navigate("detail/${game.id}") },
50                             modifier =
   Modifier.size(200.dp, 120.dp),
51                             shape =
   RoundedCornerShape(12.dp)
52                         ) {
53
   Image(painterResource(game.imageRes), null, contentScale =
   ContentScale.Crop, modifier = Modifier.fillMaxSize())
```

```
                        }
                    }
                }
                Spacer(Modifier.height(24.dp))
            }
            items(GameData.gameList) { game ->
                GameListItem(game) { navController.navigate("detail/${game.id}") }
            }
        }
    }
}

@Composable
fun GameListItem(game: Game, onDetail: () -> Unit) {
    val ctx = LocalContext.current
    val config = LocalConfiguration.current

    val lang = config.locales.get(0).language
    val isId = lang == "in" || lang == "id"

    Card(
        modifier = Modifier.padding(horizontal = 16.dp, vertical = 8.dp).fillMaxWidth().height(175.dp),
        shape = RoundedCornerShape(16.dp),
        colors = CardDefaults.cardColors(containerColor = Color(0xFF1A1A1A))
    ) {
        Row(Modifier.padding(12.dp).fillMaxSize()) {
            Image(
                painterResource(game.imageRes), null,
                modifier = Modifier.size(100.dp, 150.dp).clip(RoundedCornerShape(12.dp)),
                contentScale = ContentScale.Crop
            )
            Column(Modifier.padding(start = 12.dp).fillMaxHeight()) {
                Row(Modifier.fillMaxWidth(), horizontalArrangement = Arrangement.SpaceBetween) {
                    Text(game.title, fontWeight = FontWeight.Bold, maxLines = 1, overflow = TextOverflow.Ellipsis, modifier = Modifier.weight(1f))
                    Text(game.year, style = MaterialTheme.typography.bodySmall, color = Color.Gray)
                }
                Text(
```

```
                    text = if (isId) game.plotId else game.plotEn,
                    style = MaterialTheme.typography.bodySmall, color = Color.LightGray,
                    maxLines = 2, overflow = TextOverflow.Ellipsis, modifier = Modifier.padding(top = 4.dp), lineHeight = 16.sp
                )
                Spacer(Modifier.weight(1f))
                Row(Modifier.fillMaxWidth(), horizontalArrangement = Arrangement.End) {
                    val blue = ButtonDefaults.buttonColors(containerColor = Color(0xFF2196F3))
                    Button(onClick = { ctx.startActivity(Intent(Intent.ACTION_VIEW, game.url.toUri())) }, colors = blue, modifier = Modifier.height(36.dp)) {
                        Text(stringResource(R.string.btn_browser), fontSize = 11.sp)
                    }
                    Spacer(Modifier.width(8.dp))
                    Button(onClick = onDetail, colors = blue, modifier = Modifier.height(36.dp)) {
                        Text(stringResource(R.string.btn_detail), fontSize = 11.sp)
                    }
                }
            }
        }
    }
}
```

Package ui = > screen – SettingsScreen.kt

Tabel 6. Source Code SettingsScreen.kt Compose Soal 1

```
package com.example.modul3compose.ui.screen

import android.app.Activity
import android.content.res.Configuration
import androidx.activity.compose.LocalActivity
import androidx.compose.foundation.layout.*
import androidx.compose.material.icons.Icons
```

```
 8 import
   androidx.compose.material.icons.automirrored.filled.ArrowBa
   ck
 9 import androidx.compose.material3.*
10 import androidx.compose.runtime.Composable
11 import androidx.compose.ui.*
12 import androidx.compose.ui.graphics.Color
13 import androidx.compose.ui.res.stringResource
14 import androidx.compose.ui.text.font.FontWeight
15 import androidx.compose.ui.unit.dp
16 import com.example.modul3compose.R
17 import java.util.Locale
18
19 @Composable
20 fun SettingsScreen(navController:
   androidx.navigation.NavController) {
21     val act = LocalActivity.current
22     val blue = ButtonDefaults.buttonColors(containerColor =
   Color(0xFF2196F3))
23
24     Scaffold(
25         topBar = {
26             Surface(color = Color.Black, shadowElevation =
   4.dp) {
27                 Row(
28                     modifier =
   Modifier.fillMaxWidth().statusBarsPadding().height(64.dp).p
   adding(horizontal = 8.dp),
29                     verticalAlignment =
   Alignment.CenterVertically
30                 ) {
31                     IconButton(onClick = {
   navController.popBackStack() }) {
32
   Icon(Icons.AutoMirrored.Filled.ArrowBack, null, tint =
   Color.White)
33                     }
34
   Text(stringResource(R.string.title_settings), style =
   MaterialTheme.typography.titleLarge, fontWeight =
   FontWeight.Bold)
35                 }
36             }
37         }
38     ) { p ->
39         Column(
40             modifier = Modifier.fillMaxSize().padding(p),
```

```
            horizontalAlignment = Alignment.CenterHorizontally,
            verticalArrangement = Arrangement.Center
        ) {

LanguageButton(stringResource(R.string.btn_indonesia), blue) { act?.let { updateLang(it, "id") } }
            Spacer(Modifier.height(16.dp))

LanguageButton(stringResource(R.string.btn_english), blue) { act?.let { updateLang(it, "en") } }
        }
    }
}

@Composable
fun LanguageButton(text: String, colors: ButtonColors, onClick: () -> Unit) {
    Button(
        onClick = onClick,
        modifier = Modifier.fillMaxWidth().padding(horizontal = 48.dp),
        colors = colors
    ) {
        Text(text)
    }
}

@Suppress("DEPRECATION")
private fun updateLang(act: Activity, code: String) {
    val locale = Locale(code)
    Locale.setDefault(locale)
    val config = Configuration(act.resources.configuration)
    config.setLocale(locale)
    act.resources.updateConfiguration(config, act.resources.displayMetrics)
    act.recreate()
}
```

Package theme - Theme.kt

Tabel 7. Source Code Theme.kt Compose Soal 1

```
package com.example.modul3compose.ui.theme

import androidx.compose.material3.MaterialTheme
import androidx.compose.material3.darkColorScheme
```

```
import androidx.compose.runtime.Composable
import androidx.compose.ui.graphics.Color

private val DarkColorScheme = darkColorScheme(
    primary = Color.White,
    background = Color.Black,
    surface = Color(0xFF121212),
    onBackground = Color.White,
    onSurface = Color.White
)

@Composable
fun Modul3Theme(content: @Composable () -> Unit) {
    MaterialTheme(
        colorScheme = DarkColorScheme,
        content = content
    )
}
```

MainActivity.kt

Tabel 8. Source Code MainActivity.kt Compose Soal 1

```
package com.example.modul3compose

import android.os.Bundle
import androidx.activity.ComponentActivity
import androidx.activity.compose.setContent
import androidx.compose.foundation.layout.fillMaxSize
import androidx.compose.material3.MaterialTheme
import androidx.compose.material3.Surface
import androidx.compose.material3.darkColorScheme
import androidx.compose.ui.Modifier
import androidx.compose.ui.graphics.Color
import
com.example.modul3compose.ui.navigation.AppNavigation

class MainActivity : ComponentActivity() {
    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        setContent {
            MaterialTheme(
                colorScheme = darkColorScheme(
                    background = Color.Black,
                    surface = Color(0xFF121212),
                    onBackground = Color.White,
                    onSurface = Color.White
```

```
                )
            ) {
                Surface(
                    modifier = Modifier.fillMaxSize(),
                    color =
MaterialTheme.colorScheme.background
                ) {
                    AppNavigation()
                }
            }
        }
    }
}
```

XML

Package data - Game.kt

Tabel 9. Source Code Game.kt XML Soal 1

```
package com.example.modul3xml.data

data class Game(
    val id: Int,
    val title: String,
    val year: String,
    val plotId: String,
    val plotEn: String,
    val imageRes: Int,
    val url: String
)
```

Package data - GameData.kt

Tabel 10. Source Code GameData.kt XML Soal 1

```
package com.example.modul3xml.data

import com.example.modul3xml.R

object GameData {

    val gameList = listOf(
        Game(
            id = 1,
            title = “Honkai: Star Rail”,
            year = “2022”,
```

```
            plotId = “Perjalanan melintasi bintang-bintang di atas Astral Express.”,
            plotEn = “A journey across the stars aboard the Astral Express.”,
            imageRes = R.drawable.img_hsr,
            url = “https://hsr.hoyoverse.com/”
        ),
        Game(
            id = 2,
            title = “Arknights: Endfield”,
            year = “2026”,
            plotId = “Eksplorasi planet Talos-II yang berbahaya dan penuh misteri.”,
            plotEn = “Explore the dangerous and mysterious planet of Talos-II.”,
            imageRes = R.drawable.img_endfield,
            url = “https://endfield.gryphline.com/en-us”
        ),
        Game(
            id = 3,
            title = “Clair Obscur: Expedition 33”,
            year = “2025”,
            plotId = “Misi terakhir untuk menghancurkan entitas yang menghapus umat manusia.”,
            plotEn = “The final mission to destroy an entity erasing humanity.”,
            imageRes = R.drawable.img_coe33,
            url = “https://store.steampowered.com/app/1903340/Clair_Obscur_Expedition_33/”
        ),
        Game(
            id = 4,
            title = “Persona 5 Royal”,
            year = “2019”,
            plotId = “Kisah pencuri hati (Phantom Thieves) yang mengubah masyarakat.”,
            plotEn = “The story of Phantom Thieves changing corrupt society.”,
            imageRes = R.drawable.img_p5r,
            url = “https://store.steampowered.com/app/1687950/Persona_5_Royal/”
        ),
        Game(
            id = 5,
            title = “Persona 3 Reload”,
```

```
            year = “2024”,
            plotId = “Mengungkap misteri Dark Hour dan menyelamatkan dunia.”,
            plotEn = “Uncover the mystery of the Dark Hour and save the world.”,
            imageRes = R.drawable.img_p3r,
            url = “https://store.steampowered.com/app/2161700/Persona_3_Reload/”
        )
    )

    fun findGameById(gameId: Int): Game? {
        return gameList.find { it.id == gameId }
    }
}
```

Package ui => detail - DetailFragment.kt

Tabel 11. Source Code DetailFragment.kt XML Soal 1

```
package com.example.modul3xml.ui.detail

import android.os.Bundle
import android.view.LayoutInflater
import android.view.ViewGroup
import androidx.fragment.app.Fragment
import androidx.navigation.fragment.findNavController
import com.example.modul3xml.R
import com.example.modul3xml.data.GameData
import com.example.modul3xml.databinding.FragmentDetailBinding
import com.example.modul3xml.util.LanguageManager

class DetailFragment : Fragment() {

    private var _binding: FragmentDetailBinding? = null
    private val binding get() = _binding!!

    override fun onCreateView(
        inflater: LayoutInflater,
        container: ViewGroup?,
        savedInstanceState: Bundle?
    ) = FragmentDetailBinding.inflate(inflater, container, false).also {
        _binding = it
    }.root
```

```
25
26      override fun onViewCreated(view: android.view.View,
    savedInstanceState: Bundle?) {
27          super.onViewCreated(view, savedInstanceState)
28
29          setupToolbar()
30          bindGameData()
31      }
32
33      private fun setupToolbar() {
34          binding.toolbarDetail.setNavigationOnClickListener
    {
35              findNavController().popBackStack()
36          }
37      }
38
39      private fun bindGameData() {
40          val gameId = arguments?.getInt("gameId") ?: -1
41          val game = GameData.findGameById(gameId)
42
43          if (game == null) {
44              binding.toolbarDetail.title =
    getString(R.string.title_detail)
45              return
46          }
47
48          val isIndonesian =
    LanguageManager.isIndonesian(requireContext())
49
50          binding.toolbarDetail.title = game.title
51          binding.imageDetail.setImageResource(game.imageRes)
52          binding.textDetailTitle.text = game.title
53          binding.textDetailYear.text = game.year
54          binding.textDetailPlot.text = if (isIndonesian)
    game.plotId else game.plotEn
55      }
56
57      override fun onDestroyView() {
58          super.onDestroyView()
59          _binding = null
60      }
61  }
```

Package ui => home - GameListAdapter.kt

Tabel 12. Source Code GameListAdapter.kt XML Soal 1

```
package com.example.modul3xml.ui.home

import android.view.LayoutInflater
import android.view.View
import android.view.ViewGroup
import android.widget.Button
import android.widget.ImageView
import android.widget.TextView
import androidx.recyclerview.widget.RecyclerView
import com.example.modul3xml.R
import com.example.modul3xml.data.Game
import com.example.modul3xml.util.LanguageManager

class GameListAdapter(
    private val games: List<Game>,
    private val onBrowserClick: (Game) -> Unit,
    private val onDetailClick: (Game) -> Unit
) : RecyclerView.Adapter<GameListAdapter.GameViewHolder>() {

    override fun onCreateViewHolder(parent: ViewGroup, viewType: Int): GameViewHolder {
        val view = LayoutInflater.from(parent.context)
            .inflate(R.layout.item_game, parent, false)
        return GameViewHolder(view)
    }

    override fun onBindViewHolder(holder: GameViewHolder, position: Int) {
        val game = games[position]
        holder.bind(game, onBrowserClick, onDetailClick)
    }

    override fun getItemCount(): Int = games.size

    class GameViewHolder(itemView: View) : RecyclerView.ViewHolder(itemView) {

        private val imageGame: ImageView = itemView.findViewById(R.id.image_game)
        private val textTitle: TextView = itemView.findViewById(R.id.text_title)
        private val textYear: TextView = itemView.findViewById(R.id.text_year)
```

```
        private val textPlot: TextView = itemView.findViewById(R.id.text_plot)
        private val buttonBrowser: Button = itemView.findViewById(R.id.button_browser)
        private val buttonDetail: Button = itemView.findViewById(R.id.button_detail)

        fun bind(
            game: Game,
            onBrowserClick: (Game) -> Unit,
            onDetailClick: (Game) -> Unit
        ) {
            val isIndonesian = LanguageManager.isIndonesian(itemView.context)

            imageGame.setImageResource(game.imageRes)
            textTitle.text = game.title
            textYear.text = game.year
            textPlot.text = if (isIndonesian) game.plotId else game.plotEn

            buttonBrowser.setOnClickListener { onBrowserClick(game) }
            buttonDetail.setOnClickListener { onDetailClick(game) }
        }
    }
}
```

Package ui => home - HighlightAdapter.kt

Tabel 13. Source Code HighlightAdapter.kt XML Soal 1

```
package com.example.modul3xml.ui.home

import android.view.LayoutInflater
import android.view.View
import android.view.ViewGroup
import android.widget.ImageView
import androidx.recyclerview.widget.RecyclerView
import com.example.modul3xml.R
import com.example.modul3xml.data.Game

class HighlightAdapter(
    private val games: List<Game>,
    private val onItemClick: (Game) -> Unit
```

```
) :
RecyclerView.Adapter<HighlightAdapter.HighlightViewHolder>()
{

    override fun onCreateViewHolder(parent: ViewGroup,
viewType: Int): HighlightViewHolder {
        val view = LayoutInflater.from(parent.context)
            .inflate(R.layout.item_highlight_game, parent,
false)
        return HighlightViewHolder(view)
    }

    override fun onBindViewHolder(holder:
HighlightViewHolder, position: Int) {
        holder.bind(games[position]) { game ->
            onItemClick(game)
        }
    }

    override fun getItemCount(): Int = games.size

    class HighlightViewHolder(itemView: View) :
RecyclerView.ViewHolder(itemView) {
        private val imageGame: ImageView =
itemView.findViewById(R.id.image_highlight)

        fun bind(game: Game, onClick: (Game) -> Unit) {
            imageGame.setImageResource(game.imageRes)
            itemView.setOnClickListener {
                onClick(game)
            }
        }
    }
}
```

Package ui => home - HomeFragment.kt

Tabel 14. Source Code HomeFragment.kt XML Soal 1

```
package com.example.modul3xml.ui.home

import android.content.Intent
import android.net.Uri
import android.os.Bundle
import android.view.LayoutInflater
import android.view.ViewGroup
import androidx.fragment.app.Fragment
```

```
 9 import androidx.navigation.fragment.findNavController
10 import androidx.recyclerview.widget.LinearLayoutManager
11 import com.example.modul3xml.R
12 import com.example.modul3xml.data.GameData
13 import com.example.modul3xml.databinding.FragmentHomeBinding
14
15 class HomeFragment : Fragment() {
16
17     private var _binding: FragmentHomeBinding? = null
18     private val binding get() = _binding!!
19
20     override fun onCreateView(
21         inflater: LayoutInflater,
22         container: ViewGroup?,
23         savedInstanceState: Bundle?
24     ): android.view.View {
25         _binding = FragmentHomeBinding.inflate(inflater,
   container, false)
26         return binding.root
27     }
28
29     override fun onViewCreated(view: android.view.View,
   savedInstanceState: Bundle?) {
30         super.onViewCreated(view, savedInstanceState)
31
32         setupMenu()
33         setupHighlightList()
34         setupGameList()
35     }
36
37     private fun setupMenu() {
38         binding.toolbarHome.setOnMenuItemClickListener {
   item ->
39             when (item.itemId) {
40                 R.id.menu_settings -> {
41
   findNavController().navigate(R.id.action_homeFragment_to_set
   tingsFragment)
42                     true
43                 }
44                 else -> false
45             }
46         }
47
48         binding.toolbarHome.title =
   getString(R.string.app_name)
49     }
```

```
50
51      private fun setupHighlightList() {
52          binding.rvHighlight.layoutManager =
    LinearLayoutManager(requireContext(),
    LinearLayoutManager.HORIZONTAL, false)
53          binding.rvHighlight.adapter = HighlightAdapter(
54              games = GameData.gameList,
55              onItemClick = { game ->
56                  val bundle = Bundle().apply {
57                      putInt("gameId", game.id)
58                  }
59
    findNavController().navigate(R.id.action_homeFragment_to_det
    ailFragment, bundle)
60              }
61          )
62      }
63
64      private fun setupGameList() {
65          binding.rvGames.layoutManager =
    LinearLayoutManager(requireContext())
66          binding.rvGames.adapter = GameListAdapter(
67              games = GameData.gameList,
68              onBrowserClick = { game ->
69                  startActivity(Intent(Intent.ACTION_VIEW,
    Uri.parse(game.url)))
70              },
71              onDetailClick = { game ->
72                  val bundle = Bundle().apply {
73                      putInt("gameId", game.id)
74                  }
75
    findNavController().navigate(R.id.action_homeFragment_to_det
    ailFragment, bundle)
76              }
77          )
78      }
79
80      override fun onDestroyView() {
81          super.onDestroyView()
82          _binding = null
83      }
84
85      override fun onResume() {
86          super.onResume()
87          // refresh toolbar title so language changes are
    visible immediately
```

```
        _binding?.let { binding ->
            binding.toolbarHome.title = getString(R.string.app_name)
        }
    }
}
```

Package settings - SettingsFragment.kt

Tabel 15. Source Code SettingsFragment.kt XML Soal 1

```
package com.example.modul3xml.ui.settings

import android.os.Bundle
import android.view.LayoutInflater
import android.view.ViewGroup
import androidx.fragment.app.Fragment
import androidx.navigation.fragment.findNavController
import com.example.modul3xml.R
import com.example.modul3xml.databinding.FragmentSettingsBinding
import com.example.modul3xml.util.LanguageManager

class SettingsFragment : Fragment() {

    private var _binding: FragmentSettingsBinding? = null
    private val binding get() = _binding!!

    override fun onCreateView(
        inflater: LayoutInflater,
        container: ViewGroup?,
        savedInstanceState: Bundle?
    ): android.view.View {
        _binding = FragmentSettingsBinding.inflate(inflater, container, false)
        return binding.root
    }

    override fun onViewCreated(view: android.view.View, savedInstanceState: Bundle?) {
        super.onViewCreated(view, savedInstanceState)

        setupToolbar()
        setupLanguageButtons()
    }

    private fun setupToolbar() {
```

```
        binding.toolbarSettings.title = getString(R.string.title_settings)

binding.toolbarSettings.setNavigationOnClickListener {
            findNavController().popBackStack()
        }
    }

    private fun setupLanguageButtons() {
        binding.buttonIndonesia.setOnClickListener {
            LanguageManager.setLocale(requireActivity(), “id”)
            refreshTexts()
        }

        binding.buttonEnglish.setOnClickListener {
            LanguageManager.setLocale(requireActivity(), “en”)
            refreshTexts()
        }
    }

    private fun refreshTexts() {
        binding.toolbarSettings.title = getString(R.string.title_settings)
        binding.buttonIndonesia.text = getString(R.string.btn_indonesia)
        binding.buttonEnglish.text = getString(R.string.btn_english)
    }

    override fun onDestroyView() {
        super.onDestroyView()
        _binding = null
    }
}
```

Package util - LanguageManager.kt

Tabel 16. Source Code LanguageManager.kt XML Soal 1

```
package com.example.modul3xml.util

import android.app.Activity
import android.content.Context
import androidx.appcompat.app.AppCompatDelegate
import androidx.core.content.edit
```

```
7  import androidx.core.os.LocaleListCompat
8
9  object LanguageManager {
10
11     private const val PREF_NAME = "app_settings"
12     private const val KEY_LANG = "selected_language"
13
14     fun applySavedLocale(context: Context) {
15         val savedLanguage = getSavedLanguage(context)
16
   AppCompatDelegate.setApplicationLocales(LocaleListCompat.fo
   rLanguageTags(savedLanguage))
17     }
18
19     fun setLocale(activity: Activity, languageCode: String)
   {
20         saveLanguage(activity, languageCode)
21
   AppCompatDelegate.setApplicationLocales(LocaleListCompat.fo
   rLanguageTags(languageCode))
22     }
23
24     fun isIndonesian(context: Context): Boolean {
25         val language =
   context.resources.configuration.locales[0].language
26         return language == "id" || language == "in"
27     }
28
29     private fun getSavedLanguage(context: Context): String
   {
30         val prefs = context.getSharedPreferences(PREF_NAME,
   Context.MODE_PRIVATE)
31         return prefs.getString(KEY_LANG, "id") ?: "id"
32     }
33
34     private fun saveLanguage(context: Context,
   languageCode: String) {
35         val prefs = context.getSharedPreferences(PREF_NAME,
   Context.MODE_PRIVATE)
36         prefs.edit {
37             putString(KEY_LANG, languageCode)
38         }
39     }
40 }
```

MainActivity.kt

Tabel 17. Source Code MainActivity.kt XML Soal 1

```
package com.example.modul3xml

import android.os.Bundle
import androidx.appcompat.app.AppCompatActivity
import com.example.modul3xml.databinding.ActivityMainBinding
import com.example.modul3xml.util.LanguageManager

class MainActivity : AppCompatActivity() {

    private lateinit var binding: ActivityMainBinding

    override fun onCreate(savedInstanceState: Bundle?) {
        LanguageManager.applySavedLocale(this)
        super.onCreate(savedInstanceState)

        binding = ActivityMainBinding.inflate(layoutInflater)
        setContentView(binding.root)
    }
}
```

Package res => layout - activity_main.xml

Tabel 18. Source Code activity_main.xml XML Soal 1

```
<?xml version="1.0" encoding="utf-8"?>
<FrameLayout xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:app="http://schemas.android.com/apk/res-auto"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:background="@color/background_black">

    <androidx.fragment.app.FragmentContainerView
        android:id="@+id/nav_host_fragment"

android:name="androidx.navigation.fragment.NavHostFragment"
        android:layout_width="match_parent"
        android:layout_height="match_parent"
        app:defaultNavHost="true"
        app:navGraph="@navigation/nav_graph" />

</FrameLayout>
```

Package res => layout - fragment_detail.xml

Tabel 19. Source Code fragment_detail.xml XML Soal 1

```
1  <?xml version="1.0" encoding="utf-8"?>
2  <LinearLayout
   xmlns:android="http://schemas.android.com/apk/res/android"
3      xmlns:app="http://schemas.android.com/apk/res-auto"
4      android:layout_width="match_parent"
5      android:layout_height="match_parent"
6      android:fitsSystemWindows="true"
7      android:orientation="vertical"
8      android:background="@color/background_black">
9
10     <com.google.android.material.appbar.MaterialToolbar
11         android:id="@+id/toolbar_detail"
12         android:layout_width="match_parent"
13         android:layout_height="?attr/actionBarSize"
14         android:paddingTop="8dp"
15         android:background="@color/background_black"
16         app:navigationIcon="@drawable/ic_arrow_back"
17         app:navigationIconTint="@color/white"
18         app:titleTextColor="@color/white" />
19
20     <ScrollView
21         android:layout_width="match_parent"
22         android:layout_height="match_parent"
23         android:fillViewport="true">
24
25         <LinearLayout
26             android:layout_width="match_parent"
27             android:layout_height="wrap_content"
28             android:orientation="vertical"
29             android:padding="16dp">
30
31             <ImageView
32                 android:id="@+id/image_detail"
33                 android:layout_width="match_parent"
34                 android:layout_height="260dp"
35
   android:contentDescription="@string/cd_game_image"
36                 android:clipToOutline="true"
37                 android:scaleType="centerCrop"
38
   android:background="@drawable/bg_image_rounded" />
39
40             <LinearLayout
```

```
41                  android:layout_width="match_parent"
42                  android:layout_height="wrap_content"
43                  android:layout_marginTop="20dp"
44                  android:gravity="center_vertical"
45                  android:orientation="horizontal">
46
47                  <TextView
48                      android:id="@+id/text_detail_title"
49                      android:layout_width="0dp"
50                      android:layout_height="wrap_content"
51                      android:layout_weight="1"
52                      android:textColor="@color/white"
53                      android:textSize="28sp"
54                      android:textStyle="bold" />
55
56                  <TextView
57                      android:id="@+id/text_detail_year"
58                      android:layout_width="wrap_content"
59                      android:layout_height="wrap_content"
60                      android:textColor="@color/text_gray"
61                      android:textSize="22sp" />
62
63              </LinearLayout>
64
65              <TextView
66                  android:layout_width="wrap_content"
67                  android:layout_height="wrap_content"
68                  android:layout_marginTop="8dp"
69                  android:text="@string/title_plot"
70                  android:textColor="@color/white"
71                  android:textStyle="bold" />
72
73              <TextView
74                  android:id="@+id/text_detail_plot"
75                  android:layout_width="match_parent"
76                  android:layout_height="wrap_content"
77                  android:layout_marginTop="8dp"
78                  android:textColor="@color/white" />
79
80          </LinearLayout>
81
82      </ScrollView>
83
84  </LinearLayout>
```

Package res => layout - fragment_home.xml

Tabel 20. Source Code fragment_home.xml XML Soal 1

```
1  <?xml version="1.0" encoding="utf-8"?>
2  <LinearLayout
   xmlns:android="http://schemas.android.com/apk/res/android"
3      xmlns:app="http://schemas.android.com/apk/res-auto"
4      android:layout_width="match_parent"
5      android:layout_height="match_parent"
6      android:fitsSystemWindows="true"
7      android:orientation="vertical"
8      android:background="@color/background_black">
9
10     <com.google.android.material.appbar.MaterialToolbar
11         android:id="@+id/toolbar_home"
12         android:layout_width="match_parent"
13         android:layout_height="?attr/actionBarSize"
14         android:background="@color/background_black"
15         app:titleTextColor="@color/white"
16         app:menu="@menu/menu_home" />
17
18     <androidx.core.widget.NestedScrollView
19         android:layout_width="match_parent"
20         android:layout_height="match_parent"
21         android:fillViewport="true"
22         android:overScrollMode="never">
23
24         <LinearLayout
25             android:layout_width="match_parent"
26             android:layout_height="wrap_content"
27             android:orientation="vertical">
28
29             <TextView
30                 android:id="@+id/text_highlight"
31                 android:layout_width="match_parent"
32                 android:layout_height="wrap_content"
33                 android:layout_marginStart="16dp"
34                 android:layout_marginTop="16dp"
35                 android:layout_marginEnd="16dp"
36                 android:text="@string/title_highlight"
37                 android:textColor="@color/white"
38                 android:textSize="16sp"
39                 android:textStyle="bold" />
40
41             <androidx.recyclerview.widget.RecyclerView
42                 android:id="@+id/rv_highlight"
43                 android:layout_width="match_parent"
```

```
                android:layout_height="120dp"
                android:layout_marginTop="12dp"
                android:clipToPadding="false"
                android:paddingStart="16dp"
                android:paddingEnd="16dp"
                android:overScrollMode="never"
                android:nestedScrollingEnabled="false" />

            <androidx.recyclerview.widget.RecyclerView
                android:id="@+id/rv_games"
                android:layout_width="match_parent"
                android:layout_height="wrap_content"
                android:layout_marginTop="32dp"
                android:clipToPadding="false"
                android:paddingBottom="8dp"
                android:overScrollMode="never"
                android:nestedScrollingEnabled="false" />

        </LinearLayout>

    </androidx.core.widget.NestedScrollView>

</LinearLayout>
```

Package res => layout - fragment_settings.xml

Tabel 21. Source Code fragment_settings.xml XML Soal 1

```
<?xml version="1.0" encoding="utf-8"?>
<LinearLayout
xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:app="http://schemas.android.com/apk/res-auto"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:fitsSystemWindows="true"
    android:orientation="vertical"
    android:background="@color/background_black">

    <com.google.android.material.appbar.MaterialToolbar
        android:id="@+id/toolbar_settings"
        android:layout_width="match_parent"
        android:layout_height="?attr/actionBarSize"
        android:background="@color/background_black"
        app:navigationIcon="@drawable/ic_arrow_back"
        app:navigationIconTint="@color/white"
        app:titleTextColor="@color/white" />

```

```
    <LinearLayout
        android:layout_width="match_parent"
        android:layout_height="match_parent"
        android:orientation="vertical"
        android:gravity="center"
        android:paddingTop="16dp"
        android:paddingHorizontal="48dp">

        <com.google.android.material.button.MaterialButton
            android:id="@+id/button_indonesia"
            style="@style/Widget.Material3.Button"
            android:layout_width="match_parent"
            android:layout_height="wrap_content"
            android:text="@string/btn_indonesia"
            app:backgroundTint="@color/button_blue" />

        <Space
            android:layout_width="1dp"
            android:layout_height="16dp" />

        <com.google.android.material.button.MaterialButton
            android:id="@+id/button_english"
            style="@style/Widget.Material3.Button"
            android:layout_width="match_parent"
            android:layout_height="wrap_content"
            android:text="@string/btn_english"
            app:backgroundTint="@color/button_blue" />

    </LinearLayout>

</LinearLayout>
```

Package res - layout - item_game.xml

Tabel 22. Source Code item_game.xml XML Soal 1

```
<?xml version="1.0" encoding="utf-8"?>
<com.google.android.material.card.MaterialCardView
xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:app="http://schemas.android.com/apk/res-auto"
    android:layout_width="match_parent"
    android:layout_height="175dp"
    android:layout_marginStart="16dp"
    android:layout_marginEnd="16dp"
    android:layout_marginBottom="12dp"
    app:cardBackgroundColor="@color/card_dark"
    app:cardCornerRadius="16dp"
```

```
11      app:cardElevation="2dp"
12      app:strokeWidth="0dp">
13
14      <LinearLayout
15          android:layout_width="match_parent"
16          android:layout_height="match_parent"
17          android:orientation="horizontal"
18          android:padding="12dp">
19
20          <ImageView
21              android:id="@+id/image_game"
22              android:layout_width="100dp"
23              android:layout_height="150dp"
24
    android:contentDescription="@string/cd_game_image"
25              android:scaleType="centerCrop"
26              android:background="@drawable/bg_image_rounded"
27              android:clipToOutline="true" />
28
29          <LinearLayout
30              android:layout_width="0dp"
31              android:layout_height="match_parent"
32              android:layout_weight="1"
33              android:layout_marginStart="12dp"
34              android:orientation="vertical">
35
36              <LinearLayout
37                  android:layout_width="match_parent"
38                  android:layout_height="wrap_content"
39                  android:orientation="horizontal"
40                  android:gravity="center_vertical">
41
42                  <TextView
43                      android:id="@+id/text_title"
44                      android:layout_width="0dp"
45                      android:layout_height="wrap_content"
46                      android:layout_weight="1"
47                      android:textColor="@color/white"
48                      android:textStyle="bold"
49                      android:maxLines="1"
50                      android:ellipsize="end" />
51
52                  <TextView
53                      android:id="@+id/text_year"
54                      android:layout_width="wrap_content"
55                      android:layout_height="wrap_content"
56                      android:textColor="@color/text_gray"
```

```
                    android:textSize="12sp" />

            </LinearLayout>

            <TextView
                android:id="@+id/text_plot"
                android:layout_width="match_parent"
                android:layout_height="wrap_content"
                android:layout_marginTop="8dp"
                android:textColor="@color/text_light_gray"
                android:textSize="12sp"
                android:lineSpacingExtra="2dp"
                android:maxLines="2"
                android:ellipsize="end" />

            <!-- Spacer pushes the buttons to the bottom of the card (aligned with the image bottom) -->
            <Space
                android:layout_width="match_parent"
                android:layout_height="0dp"
                android:layout_weight="1" />

            <LinearLayout
                android:layout_width="match_parent"
                android:layout_height="wrap_content"
                android:gravity="bottom|end"
                android:orientation="horizontal">


<com.google.android.material.button.MaterialButton
                    android:id="@+id/button_browser"
                    style="@style/Widget.Material3.Button"
                    android:layout_width="wrap_content"
                    android:layout_height="36dp"
                    android:insetTop="0dp"
                    android:insetBottom="0dp"
                    android:minWidth="0dp"
                    android:text="@string/btn_browser"
                    android:textSize="11sp"
                    app:backgroundTint="@color/button_blue" />

                <Space
                    android:layout_width="8dp"
                    android:layout_height="1dp" />

```

```

<com.google.android.material.button.MaterialButton
                    android:id="@+id/button_detail"
                    style="@style/Widget.Material3.Button"
                    android:layout_width="wrap_content"
                    android:layout_height="36dp"
                    android:insetTop="0dp"
                    android:insetBottom="0dp"
                    android:minWidth="0dp"
                    android:text="@string/btn_detail"
                    android:textSize="11sp"
                    app:backgroundTint="@color/button_blue"
/>

            </LinearLayout>

        </LinearLayout>

    </LinearLayout>

</com.google.android.material.card.MaterialCardView>
```

Package res => layout - item_highlight_game.xml

Tabel 23. Source Code item_hightlight_game.xml XML Soal 1

```
<?xml version="1.0" encoding="utf-8"?>
<com.google.android.material.card.MaterialCardView
xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:app="http://schemas.android.com/apk/res-auto"
    android:layout_width="200dp"
    android:layout_height="120dp"
    android:layout_marginEnd="12dp"
    app:cardBackgroundColor="@color/card_dark"
    app:cardCornerRadius="12dp"
    app:cardElevation="2dp"
    app:strokeWidth="0dp">

    <ImageView
        android:id="@+id/image_highlight"
        android:layout_width="match_parent"
        android:layout_height="match_parent"
        android:scaleType="centerCrop"
        android:contentDescription="@string/cd_game_image"
/>

</com.google.android.material.card.MaterialCardView>
```

Package res => menu - menu_home.xml

Tabel 24. Source Code menu_home.xml XML Soal 1

```
<?xml version="1.0" encoding="utf-8"?>
<menu
xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:app="http://schemas.android.com/apk/res-auto">

    <item
        android:id="@+id/menu_settings"
        android:icon="@drawable/ic_settings"
        android:title="@string/title_settings"
        app:showAsAction="ifRoom" />

</menu>
```

Package res => navigation - nav_graph.xml

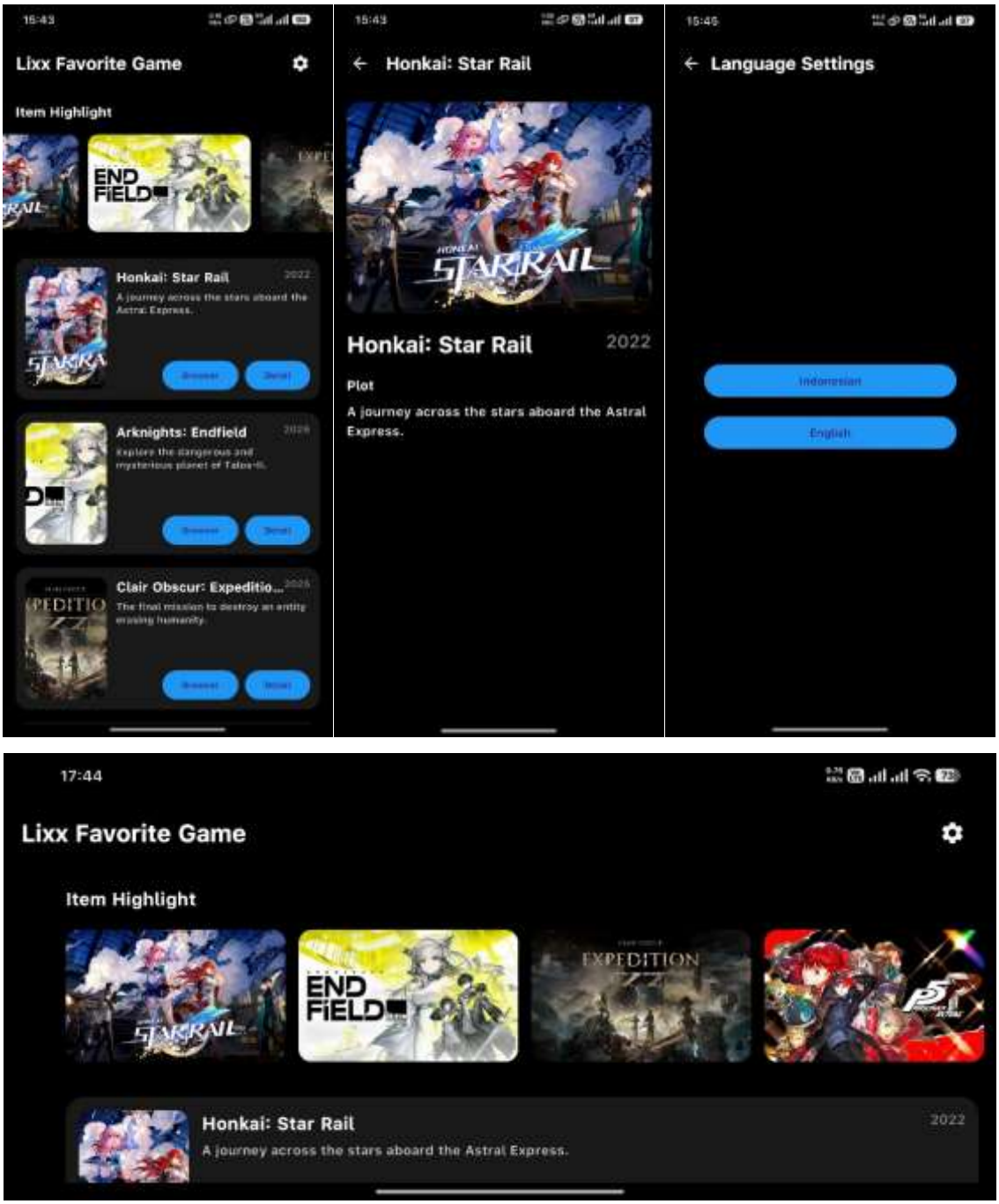
Tabel 25. Source Code nav_graph.xml XML Soal 1

```
<?xml version="1.0" encoding="utf-8"?>
<navigation
xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:app="http://schemas.android.com/apk/res-auto"
    android:id="@+id/nav_graph"
    app:startDestination="@id/homeFragment">

    <fragment
        android:id="@+id/homeFragment"

android:name="com.example.modul3xml.ui.home.HomeFragment"
        android:label="Home">
        <action

android:id="@+id/action_homeFragment_to_detailFragment"
            app:destination="@id/detailFragment"
            app:enterAnim="@anim/slide_in_right"
            app:exitAnim="@anim/slide_out_left"
            app:popEnterAnim="@anim/slide_in_left"
            app:popExitAnim="@anim/slide_out_right" />
        <action

android:id="@+id/action_homeFragment_to_settingsFragment"
            app:destination="@id/settingsFragment"
            app:enterAnim="@anim/slide_in_right"
            app:exitAnim="@anim/slide_out_left"
```

```
23              app:popEnterAnim="@anim/slide_in_left"
24              app:popExitAnim="@anim/slide_out_right" />
25      </fragment>
26
27      <fragment
28          android:id="@+id/detailFragment"
29
    android:name="com.example.modul3xml.ui.detail.DetailFragmen
    t"
30          android:label="Detail" />
31
32      <fragment
33          android:id="@+id/settingsFragment"
34
    android:name="com.example.modul3xml.ui.settings.SettingsFra
    gment"
35          android:label="Settings" />
36
37  </navigation>
```

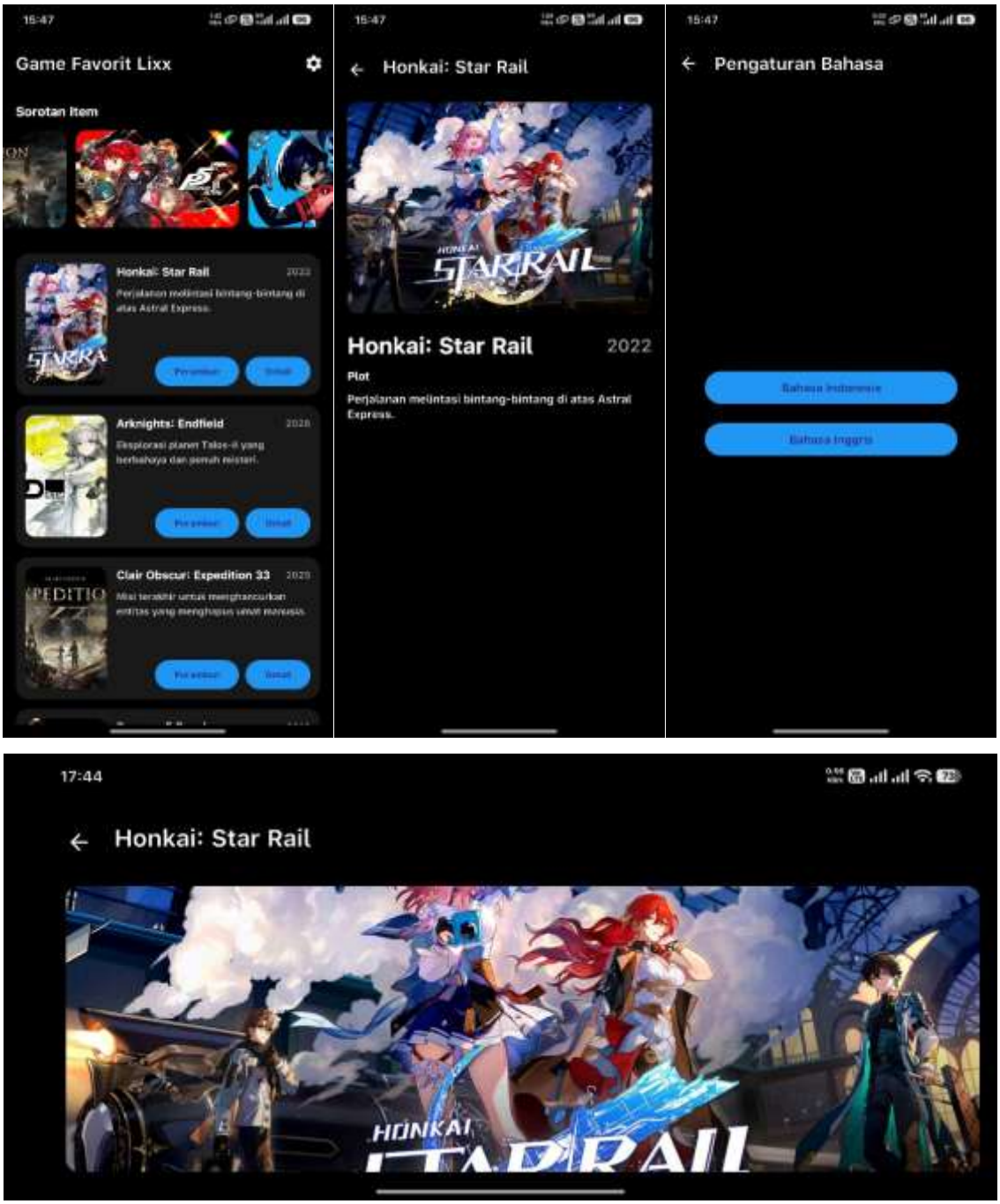
B. Output Program

COMPOSE



Gambar 4. Screenshot Hasil Jawaban Soal 1 Compose

XML



Gambar 5. Screenshot Hasil Jawaban Soal 1 XML

C. Pembahasan

COMPOSE

Package data - GameData.kt

File GameData.kt pada package ‘com.example.modul3compose.data’ dibuat untuk menyimpan dummy data statis yang akan digunakan di dalam aplikasi. Pada bagian atas kode, terdapat import untuk ‘com.example.modul3compose.R’ yang sangat penting agar kode dapat mendeteksi dan membaca aset gambar yang tersimpan di dalam folder drawable Android. Selain itu, ada juga import untuk kelas model ‘Game’ yang akan digunakan sebagai struktur dari data yang dimasukkan.

Di dalam object tersebut, dibuat sebuah variabel bernama ‘gameList’ menggunakan keyword ‘val’. Variabel ini menampung elemen data menggunakan fungsi ‘listOf’, di mana isi dari list tersebut adalah sekumpulan objek dari data class ‘Game’.

Untuk setiap objek ‘Game’ yang ada di dalam list, datanya diisi secara berurutan sesuai dengan parameter yang diminta oleh model kelasnya. Argumen yang dimasukkan meliputi ID unik bertipe angka (1, 2, 3, dst.), judul game, tahun rilis, deskripsi bahasa Indonesia, deskripsi bahasa Inggris, ID pemanggilan resource gambar seperti ‘R.drawable.img_hsr’, dan diakhiri dengan URL web resmi game tersebut

Package model - Game.kt

File Game.kt pada package ‘com.example.modul3compose.model’ menggunakan deklarasi ‘data class’ untuk membuat cetakan atau struktur dasar dari objek game. Di dalam kelas ‘Game’, terdapat tujuh parameter yang dideklarasikan langsung di dalam konstruktor utama menggunakan keyword ‘val’. Atribut pertama adalah ‘id’ bertipe ‘Int’ yang bertugas sebagai identitas unik untuk setiap game. Kemudian ada atribut ‘title’ untuk judul, ‘year’ untuk tahun rilis, ‘plotId’ untuk deskripsi bahasa Indonesia, ‘plotEn’ untuk deskripsi bahasa Inggris, dan ‘url’ untuk tautan halaman web. Kelima atribut ini menggunakan tipe data ‘String’ karena difungsikan untuk menyimpan teks.

Selain itu, terdapat atribut ‘imageRes’ yang sengaja dideklarasikan dengan tipe data ‘Int’. Atribut ini tidak bertipe String karena fungsinya bukan untuk menyimpan nama file gambar, melainkan untuk menampung ID resource dari aset gambar yang tersimpan di dalam folder drawable.

Package ui => navigation - AppNavigation.kt

File AppNavigation.kt yang berada di dalam package ‘com.example.modul3compose.ui.navigation’ berfungsi sebagai pusat pengatur rute dan alur perpindahan antarlayar di dalam aplikasi. Pada fungsi ‘AppNavigation’ yang ditandai dengan anotasi ‘@Composable’, langkah pertama yang dilakukan adalah menginisialisasi variabel ‘navController’ menggunakan fungsi ‘rememberNavController()’. Variabel ini berperan sebagai pengontrol utama yang akan mengingat status riwayat navigasi (back stack) dari pengguna. Setelah itu, dipanggil komponen ‘NavHost’ yang bertindak sebagai wadah tempat layar-layar yang akan dirender. Pada komponen ini, variabel ‘navController’ disematkan dan parameter ‘startDestination’ diisi dengan nilai string “home”. Ini berarti saat aplikasi pertama kali dijalankan, rute yang langsung dimuat adalah halaman utama.

Di dalam blok ‘NavHost’, daftar rute aplikasi didaftarkan menggunakan fungsi ‘composable’. Rute pertama didefinisikan dengan string “home”, yang di dalamnya akan mengeksekusi dan menampilkan ‘HomeScreen’. Rute kedua adalah “detail/{gameId}”, yang dirancang khusus untuk menerima parameter dinamis berupa ID dari game yang diklik oleh pengguna. Di dalam rute ini, kita menangkap nilai argumen yang dikirimkan menggunakan ‘backStackEntry.arguments?.getString(“gameId”)’, lalu langsung mengonversinya menjadi tipe integer menggunakan fungsi pemrosesan aman ‘toIntOrNull()’. Nilai ID yang sudah didapat ini kemudian dilempar ke komponen ‘DetailScreen’ agar halaman tersebut bisa memuat data spesifik sesuai game yang dipilih. Rute terakhir yang didaftarkan adalah “settings”, yang bertugas memanggil komponen ‘SettingsScreen’.

Package ui => screen - DetailScreen.kt

Fungsi ‘DetailScreen’ pada file ini bertugas merender halaman detail antarmuka untuk game yang dipilih. Fungsi ini menerima ‘navController’ untuk menangani alur navigasi dan ‘gameId’ bertipe integer sebagai penanda unik data mana yang harus ditarik. Di baris awal, program langsung mencari spesifikasi data game dengan mencocokkan ‘gameId’ ke dalam koleksi ‘GameData.gameList’ menggunakan fungsi ‘find’. Setelah datanya didapat, program mendeteksi bahasa bawaan dari sistem perangkat lewat ‘LocalConfiguration.current.locales’. Hasilnya dievaluasi ke dalam variabel boolean ‘isId’ yang akan bernilai true jika sistem

memakai bahasa Indonesia (“in” atau “id”). Variabel ini disiapkan untuk menentukan bahasa deskripsi cerita yang akan di-render nanti.

Untuk kerangka dasar halamannya, digunakan komponen ‘Scaffold’. Pada bagian ‘topBar’, dibuat bar navigasi atas berlatar hitam menggunakan ‘Surface’ yang diberi sedikit efek bayangan melalui ‘shadowElevation’. Di dalamnya disisipkan tata letak horizontal berupa ‘Row’ yang menampung tombol panah kembali (‘IconButton’) dan teks judul. Ketika tombol kembali ditekan, aksi klik akan memicu perintah ‘navController.popBackStack()’ yang bertugas menghapus halaman detail ini dari tumpukan memori dan mengembalikan layar ke halaman sebelumnya. Tepat di sebelahnya, judul game ditampilkan menggunakan Elvis operator (‘?:’), sehingga seandainya data judul bernilai null, layar otomatis akan menampilkan teks default “Detail”.

Masuk ke area konten utama, proses rendering elemen UI dibungkus menggunakan fungsi safe call ‘let’ pada objek ‘game’. Seluruh detail informasi disusun secara vertikal ke bawah menggunakan ‘Column’. Agar halaman tetap nyaman dibaca dan bisa digeser ketika teks ceritanya panjang, ‘Column’ tersebut dipasangi modifier ‘verticalScroll(rememberScrollState())’.

Pada urutan paling atas konten, komponen ‘Image’ memuat aset gambar sampul game melalui ‘painterResource’. Di bawah gambar, terdapat komponen ‘Row’ dengan pengaturan ‘Arrangement.SpaceBetween’ untuk meletakkan teks judul dan tahun rilis. Teks judul sengaja diberi modifier ‘weight(1f)’ supaya elemen tersebut memakan semua sisa ruang horizontal yang kosong, yang membuat teks tahun rilis terdorong rapi ke ujung kanan layar. Terakhir, program menampilkan teks alur cerita, di mana pemanggilan datanya antara ‘plotId’ atau ‘plotEn’ dipilih secara kondisional berdasarkan status boolean ‘isId’ yang sudah dideteksi di awal.

Package ui => screen - HomeScreen.kt

Fungsi ‘HomeScreen’ pada antarmuka ini bertugas menyusun tata letak halaman utama aplikasi dengan memanfaatkan kerangka ‘Scaffold’. Pada slot ‘topBar’, program membuat bar navigasi atas berlatar hitam menggunakan ‘Surface’. Di dalamnya, disematkan komponen ‘Row’ yang mengatur tata letak horizontal untuk teks judul aplikasi yang diambil lewat ‘stringResource’ dan sebuah tombol ikon ‘Settings’. Teks judul tersebut dipasangkan modifier

‘weight(1f)’ sehingga sisa ruang kosong akan terisi penuh, yang efeknya mendorong ikon pengaturan ke sudut kanan layar. Apabila ikon tersebut diklik, ‘navController’ akan menjalankan perintah navigasi menuju rute “settings”.

Untuk area konten utamanya, program menggunakan ‘LazyColumn’ yang membungkus keseluruhan sisa layar. Di dalam ‘LazyColumn’, pada blok ‘item’ yang pertama, layar menampilkan bagian sorotan (highlight). Bagian ini terdiri dari judul teks dan komponen ‘LazyRow’ untuk memunculkan daftar gulir secara horizontal. Melalui fungsi ‘items’, program melakukan iterasi (perulangan) pada himpunan data ‘GameData.gameList’ dan mencetaknya menjadi kumpulan ‘Card’ berisi ‘Image’ sampul game. Jika salah satu ‘Card’ pada daftar horizontal ini ditekan, program akan melakukan navigasi ke rute “detail/{game.id}”.

Di bawah bagian sorotan tersebut, masih di dalam ‘LazyColumn’, program kembali memanggil fungsi ‘items’ untuk melakukan perulangan pada data yang sama. Namun kali ini, data yang dibaca dilempar sebagai argumen ke fungsi komponen kustom bernama ‘GameListItem’. Bersamaan dengan data tersebut, dilempar juga sebuah fungsi lambda berisi perintah ‘navController.navigate’ agar tombol detail di dalam item tersebut nantinya bisa melakukan perpindahan layar.

Beralih ke fungsi ‘GameListItem’. Pada baris awal fungsi ini, program memanggil ‘LocalContext.current’ yang disimpan dalam variabel ‘ctx’ untuk kebutuhan eksekusi Intent nantinya, dan ‘LocalConfiguration’ untuk mendeteksi bahasa sistem ke dalam variabel ‘isId’ (seperti logika pada halaman detail). Seluruh elemen dalam komponen ini dibungkus menggunakan ‘Card’ berlatar warna kustom gelap (‘Color(0xFF1A1A1A)’).

Struktur di dalam ‘Card’ disusun menyamping lewat ‘Row’. Sisi kiri dialokasikan untuk memuat ‘Image’ gambar game, sedangkan sisi kanan diisi komponen ‘Column’ agar teks judul, tahun, dan plot cerita bisa tersusun urut dari atas ke bawah. Untuk menjaga antarmuka tetap rapi, teks judul diberi batas ‘maxLines = 1’ dan teks plot dibatasi ‘maxLines = 2’. Jika teks melebihi batas tersebut, sistem akan memotongnya dan menambahkan titik tiga di bagian ujung menggunakan fungsi ‘TextOverflow.Ellipsis’.

Pada bagian paling bawah di dalam ‘Column’ sisi kanan tersebut, program meletakkan ‘Spacer’ dengan modifier ‘weight(1f)’ yang berfungsi sebagai pendorong sisa elemen ke dasar bidang. Di bawahnya, terdapat tata letak ‘Row’ dengan perataan ‘Arrangement.End’ untuk

menampung dua buah tombol (‘Button’) berwarna biru. Tombol pertama (“Browser”) diinstruksikan untuk menjalankan fungsi ‘ctx.startActivity’ yang membawa implicit intent berjenis ‘Intent.ACTION_VIEW’ guna membuka tautan URL game di browser perangkat. Sementara itu, tombol kedua (“Detail”) berfungsi untuk mengeksekusi parameter lambda ‘onDetail’ yang telah diteruskan dari ‘HomeScreen’ tadi untuk berpindah ke halaman detail.

Package ui = > screen – SettingsScreen.kt

Fungsi ‘SettingsScreen’ pada file ini fokus utamanya adalah menyediakan fitur untuk mengubah bahasa aplikasi. Di baris awal fungsi, program memanggil ‘LocalActivity.current’ ke dalam variabel ‘act’ guna mendapatkan akses atau referensi dari activity Android yang sedang berjalan.

Untuk kerangka antarmukanya, digunakan komponen ‘Scaffold’. Pada slot ‘topBar’, program membuat bar navigasi atas berlatar hitam melalui komponen ‘Surface’. Di dalamnya disusun sebuah tata letak ‘Row’ yang menampung tombol panah kembali (‘IconButton’) dan teks judul. Ketika tombol kembali ditekan, program memanggil fungsi ‘navController.popBackStack()’ untuk mengeluarkan halaman pengaturan ini dari memori dan mengembalikan layar ke halaman sebelumnya.

Masuk ke area konten di dalam ‘Scaffold’, program menyusun elemen secara vertikal dan memosisikannya tepat di tengah-tengah layar menggunakan komponen ‘Column’ dengan atribut ‘horizontalAlignment = Alignment.CenterHorizontally’ serta ‘verticalArrangement = Arrangement.Center’. Di dalam kolom tersebut, program memanggil komponen kustom ‘LanguageButton’ sebanyak dua kali untuk opsi bahasa Indonesia dan bahasa Inggris, diselingi oleh jarak kosong menggunakan ‘Spacer’. Saat tombol ditekan, aksi klik akan mengeksekusi blok lambda yang berisi fungsi safe call ‘act?.let’, lalu memanggil fungsi ‘updateLang’ dengan mengirimkan parameter activity dan kode bahasa (“id” atau “en”).

Fungsi ‘LanguageButton’ di bawahnya merupakan komponen reusable yang dipisah khusus agar penulisan kode tidak berulang. Fungsi ini pada dasarnya membungkus komponen ‘Button’ standar Compose, melebarkannya dengan modifier ‘fillMaxWidth()’, dan memberikan jarak margin di sisi kiri dan kanan. Teks, warna, dan aksi klik dari tombol ini diatur secara dinamis berdasarkan parameter yang di-passing dari luar.

Bagian terakhir adalah fungsi private ‘updateLang’ yang memuat logika under-the-hood untuk mengganti bahasa aplikasi. Fungsi ini ditandai dengan anotasi ‘@Suppress(“DEPRECATION”)’ untuk mencegah warning atau peringatan dari compiler terkait penggunaan metode pembaruan konfigurasi yang sudah dianggap usang (deprecated) pada API Android terbaru. Di dalam blok ini, program membuat instansiasi ‘Locale’ baru sesuai kode bahasa yang dipilih dan menjadikannya sebagai bahasa default. Kemudian, program memodifikasi objek ‘Configuration’ bawaan perangkat untuk menggunakan ‘Locale’ baru tersebut dan menerapkannya ke dalam resources aplikasi.

Package theme - Theme.kt

File Theme.kt pada package ‘com.example.modul3compose.ui.theme’ difungsikan sebagai pengatur skema warna dan tema dasar untuk keseluruhan antarmuka aplikasi. Pada bagian atas kode, dideklarasikan sebuah variabel private bernama ‘DarkColorScheme’ yang menampung konfigurasi palet warna menggunakan fungsi ‘darkColorScheme’. Pada konfigurasi ini, skema warna antarmuka sengaja diatur agar selalu tampil dalam mode gelap (dark mode) secara statis. Warna latar belakang utama (‘background’) ditetapkan menjadi hitam pekat (‘Color.Black’), sedangkan elemen penampang atau wadah seperti komponen kartu (‘surface’) diberi warna abu-abu sangat gelap dengan representasi kode hex (‘Color(0xFF121212)’). Agar teks dan elemen visual lainnya tetap kontras dan terbaca jelas, atribut ‘onBackground’ dan ‘onSurface’ diatur menjadi putih (‘Color.White’).

Selanjutnya, terdapat fungsi ‘Modul3Theme’ yang ditandai dengan anotasi ‘@Composable’. Fungsi ini bertindak sebagai komponen pembungkus utama (wrapper) bagi arsitektur UI aplikasi. Fungsi tersebut menerima sebuah parameter lambda bernama ‘content’, yang merepresentasikan seluruh hierarki elemen antarmuka yang akan disisipkan di dalamnya. Di dalam blok tersebut, program memanggil komponen ‘MaterialTheme’ bawaan dari pustaka Jetpack Compose dan menyematkan variabel ‘DarkColorScheme’ ke dalam parameter ‘colorScheme’. Kemudian, parameter ‘content’ dieksekusi tepat di dalamnya.

MainActivity.kt

Pada bagian ‘onCreate’, program memanggil fungsi ‘setContent’ yang bertindak sebagai jembatan utama untuk mulai menggambar atau merender komponen antarmuka Jetpack

Compose ke layar perangkat. Di dalam blok ‘setContent’ tersebut, program secara langsung menyematkan komponen ‘MaterialTheme’ guna mengatur skema warna aplikasi. Pengaturan warnanya didefinisikan langsung menggunakan fungsi ‘darkColorScheme’, di mana warna latar belakang utama (‘background’) diatur menjadi hitam pekat (‘Color.Black’), warna elemen penampang (‘surface’) menggunakan abu-abu sangat gelap (‘Color(0xFF121212)’), serta warna teks yang berada di atas latar dan penampang tersebut (‘onBackground’ dan ‘onSurface’) ditetapkan menjadi putih agar visualnya tetap kontras dan jelas terbaca.

Setelah kerangka tema terbentuk, program memanggil komponen ‘Surface’ yang berfungsi sebagai kanvas atau wadah dasar untuk halaman aplikasi. Kanvas ini diberikan ‘Modifier.fillMaxSize()’ agar ukurannya merentang penuh menutupi seluruh dimensi layar, dan atribut warnanya dihubungkan ke ‘MaterialTheme.colorScheme.background’. Terakhir, program akan mengeksekusi fungsi ‘AppNavigation()’. Pemanggilan komponen ini menjadi titik awal (entry point) aplikasi, di mana kendali antarmuka selanjutnya diserahkan sepenuhnya kepada sistem navigasi untuk memuat dan mengatur perpindahan halaman-halaman yang ada.

XML

Package data - Game.kt

File Game.kt yang berada di dalam package ‘com.example.modul3xml.data’ menggunakan deklarasi ‘data class’ untuk membuat cetakan atau struktur dasar dari objek game. Sama seperti pada versi Compose sebelumnya, penggunaan ‘data class’ di Kotlin sangat efisien karena memang dirancang khusus untuk menampung data; sistem akan otomatis menyediakan fungsi-fungsi pendukung di latar belakang (seperti equals, hashCode, dan toString), sehingga kita cukup mendefinisikan atribut-atribut utamanya saja tanpa perlu repot menulis kode pelengkap secara manual.

Di dalam kelas ‘Game’ ini, terdapat tujuh parameter yang dideklarasikan langsung di dalam konstruktor utama menggunakan keyword ‘val’. Penggunaan ‘val’ memastikan bahwa setiap atribut bersifat konstan atau read-only, yang artinya nilai data tidak bisa dimodifikasi lagi setelah objek game selesai diinstansiasi. Atribut pertama adalah ‘id’ bertipe ‘Int’ yang bertugas sebagai identitas unik untuk setiap game. Kemudian ada atribut ‘title’ untuk judul, ‘year’ untuk tahun rilis, ‘plotId’ untuk deskripsi bahasa Indonesia, ‘plotEn’ untuk deskripsi

bahasa Inggris, dan ‘url’ untuk tautan halaman web referensi. Kelima atribut ini menggunakan tipe data ‘String’ karena difungsikan untuk menyimpan teks.

Selain itu, terdapat atribut ‘imageRes’ yang sengaja dideklarasikan dengan tipe data ‘Int’. Atribut ini tidak bertipe String karena fungsinya bukan untuk menyimpan nama file gambar, melainkan untuk menampung ID resource dari aset gambar yang tersimpan di dalam folder drawable. Di dalam sistem Android, setiap aset yang dipanggil dan diikat ke antarmuka XML (misalnya menggunakan format ‘R.drawable.nama_gambar’) otomatis dipetakan dan dikenali oleh sistem sebagai nilai angka integer unik.

Package data - GameData.kt

File GameData.kt pada package ‘com.example.modul3xml.data’ berfungsi sebagai tempat penyimpanan data statis (dummy data) lokal untuk aplikasi. Pada bagian atas kode, terdapat import kelas ‘R’ yang wajib disertakan agar program dapat mengenali referensi aset gambar dari folder drawable.

Di dalam object tersebut, dideklarasikan sebuah variabel bernama ‘gameList’ menggunakan keyword ‘val’. Variabel ini menampung koleksi data menggunakan fungsi ‘listOf’, di mana isi dari daftar tersebut adalah sekumpulan objek hasil instansiasi dari data class ‘Game’.Penulisan nilai pada konstruktor objek di sini menggunakan teknik named arguments (seperti ‘id = 1’, ‘title = “Honkai: Star Rail”’, dsb.).

Selain daftar game, di bagian bawah object ini ditambahkan sebuah fungsi pembantu bernama ‘findGameById’. Fungsi ini menerima parameter ‘gameId’ bertipe integer dan dirancang untuk mengembalikan tipe data ‘Game?’ (nullable). Di dalam blok fungsinya, program menjalankan perintah ‘find’ pada himpunan data ‘gameList’ untuk menyeleksi dan mencari elemen yang nilai atribut ‘id’-nya sama persis dengan parameter ‘gameId’ yang dilempar. Jika data yang dicari ada, fungsi akan mengembalikan objek game tersebut secara utuh, namun jika tidak ada yang cocok, fungsi akan otomatis mengembalikan nilai null. Fungsi pencarian ini dipersiapkan secara khusus untuk mempermudah halaman detail (Activity lain) saat perlu menarik spesifikasi data game berdasarkan ID yang nantinya dikirimkan melalui Intent.

Package ui => detail - DetailFragment.kt

File DetailFragment.kt pada package ‘com.example.modul3xml.ui.detail’ mewakili halaman antarmuka detail yang dibangun menggunakan struktur Fragment dan sistem View Binding. Pada bagian awal kelas, dideklarasikan dua variabel untuk mengelola binding antarmuka, yaitu ‘_binding’ yang bersifat nullable dan ‘binding’ yang merupakan referensi getter non-null. Pada fungsi ‘onCreateView’, proses inflate atau perangkaian tata letak visual dari file XML ke dalam objek Kotlin dilakukan menggunakan ‘FragmentDetailBinding.inflate’. Hasil inflate tersebut disematkan ke variabel ‘_binding’ lewat fungsi scope ‘also’, lalu mengembalikan elemen ‘root’ ke sistem agar halamannya tampil di layar.

Setelah komponen tampilan selesai dibuat, eksekusi program masuk ke dalam siklus hidup ‘onViewCreated’. Di blok ini, program memanggil dua fungsi utama secara berurutan: ‘setupToolbar()’ dan ‘bindGameData()’. Pada fungsi ‘setupToolbar’, program memasangkan listener aksi klik pada tombol panah kembali yang berada di komponen ‘toolbarDetail’. Ketika tombol tersebut ditekan, perintah ‘findNavController().popBackStack()’ akan tereksekusi untuk menghapus Fragment detail ini dari riwayat navigasi (back stack) dan mengembalikan pengguna secara otomatis ke layar daftar game.

Fungsi ‘bindGameData’ berperan sebagai inti logika penarikan dan pendistribusian data ke layar. Pertama, program mengambil data angka ID yang dilempar dari halaman sebelumnya menggunakan ‘arguments?.getInt(“gameId”)’. Jika ID kosong atau gagal ditangkap, Elvis operator akan memberikan nilai default -1. Angka ID tersebut kemudian diumpankan ke fungsi ‘GameData.findGameById’ untuk menarik spesifikasi objek game secara utuh. Terdapat blok pengecekan kondisional (if) di bawahnya; jika objek game ternyata tidak ditemukan atau bernilai null, maka judul ‘toolbarDetail’ akan diisi teks default dari string resource, dan sisa kode di bawahnya tidak akan dieksekusi karena diputus oleh perintah ‘return’.

Jika data game berhasil ditemukan, program akan memanggil kelas utilitas ‘LanguageManager.isIndonesian(requireContext())’ untuk mendeteksi apakah sistem sedang memakai bahasa Indonesia. Setelah status bahasanya diketahui, data atribut objek game mulai diikat ke komponen antarmuka XML. Teks judul dipasangkan ke ‘toolbarDetail.title’ dan ‘textDetailTitle’, tahun rilis disisipkan ke ‘textDetailYear’, dan resource gambar sampul

dimuat ke dalam komponen ‘imageDetail’ melalui metode ‘setImageResource’. Pada komponen ‘textDetailPlot’, pemanggilan teks deskripsi antara ‘plotId’ atau ‘plotEn’ diputuskan secara dinamis melalui percabangan inline if-else berdasarkan hasil evaluasi bahasa sistem. Terakhir, saat Fragment dihancurkan dan masuk ke siklus ‘onDestroyView’, variabel ‘_binding’ sengaja dikosongkan (null) untuk memastikan sisa referensi view di memori benar-benar terhapus.

Package ui => home - GameListAdapter.kt

Kelas ‘GameListAdapter’ pada file ini bertugas sebagai jembatan atau adapter untuk menghubungkan koleksi data game dengan komponen antarmuka ‘RecyclerView’ agar bisa ditampilkan menjadi daftar gulir vertikal. Pada konstruktor utamanya, kelas ini menerima tiga parameter: variabel ‘games’ yang berisi himpunan data berbentuk list, serta dua fungsi lambda yaitu ‘onBrowserClick’ dan ‘onDetailClick’ yang secara spesifik disiapkan untuk menangani event klik tombol pada masing-masing baris elemen data.

Karena mewarisi ‘RecyclerView.Adapter’, terdapat tiga fungsi wajib yang di-override untuk mengelola mekanisme daftarnya. Fungsi ‘onCreateViewHolder’ bertanggung jawab untuk melakukan inflate file tata letak XML kustom ‘R.layout.item_game’ menjadi sebuah objek View nyata di layar, lalu membungkusnya dan mengembalikannya ke dalam bentuk ‘GameViewHolder’. Selanjutnya, fungsi ‘getItemCount’ digunakan untuk mengembalikan nilai ukurang dari pemanggilan ‘games.size’ agar sistem RecyclerView mengetahui total panjang baris yang perlu dimuat. Lalu terdapat fungsi ‘onBindViewHolder’, yang beroperasi dengan mengambil data game dari list berdasarkan indeks urutan ‘position’, kemudian melempar objek data tersebut beserta kedua fungsi lambda tadi ke dalam metode ‘bind’ milik ViewHolder.

Di bagian bawah, didefinisikan kelas bersarang (inner class) ‘GameViewHolder’ yang bertindak sebagai penyimpan referensi dari komponen-komponen antarmuka XML. Pada saat kelas ini dibuat, program langsung memetakan elemen UI satu per satu ke dalam variabel lokal (seperti ‘image_game’ ke ‘imageGame’, ‘text_title’ ke ‘textTitle’) menggunakan fungsi bawaan ‘findViewById’.

Inti dari proses penempelan data terjadi di dalam fungsi ‘bind’. Program akan mengawali blok ini dengan mendeteksi status bahasa perangkat lewat kelas utilitas

‘LanguageManager.isIndonesian(itemView.context)’. Setelah itu, program mulai memasukkan nilai atribut dari objek ‘game’ ke komponen visual: menyisipkan resource gambar ke ‘imageGame’ lewat ‘setImageResource’, mencetak judul di ‘textTitle’, dan tahun di ‘textYear’. Khusus untuk ‘textPlot’, kalimat yang dimuat akan dipilih antara atribut ‘plotId’ atau ‘plotEn’ melalui evaluasi inline if yang menyesuaikan dengan kondisi bahasa sistem yang telah dideteksi. Sebagai langkah akhir penyiapan, fungsi ‘setOnClickListener’ disematkan pada ‘buttonBrowser’ dan ‘buttonDetail’. Ketika diklik, tombol tersebut akan langsung memanggil fungsi lambda masing-masing sambil membawa spesifik objek ‘game’ tersebut ke komponen induk tempat adapter ini dipanggil.

Package ui => home - HighlightAdapter.kt

Kelas ‘HighlightAdapter’ pada kode ini bertugas sebagai penghubung (adapter) antara koleksi data game dengan komponen antarmuka ‘RecyclerView’ yang dikhususkan untuk menampilkan daftar sorotan (highlight). Pada konstruktor utamanya, kelas ini menerima dua parameter: variabel ‘games’ yang berisi himpunan data berwujud list, serta sebuah fungsi lambda ‘onItemClick’ yang disiapkan untuk menangani interaksi saat salah satu item ditekan oleh pengguna.

Karena mewarisi kelas ‘RecyclerView.Adapter’, terdapat tiga fungsi wajib yang di-override untuk mengendalikan mekanisme daftarnya. Fungsi ‘onCreateViewHolder’ bertanggung jawab untuk melakukan inflate atau memuat tata letak visual dari file XML ‘R.layout.item_highlight_game’ menjadi sebuah objek View secara utuh, yang kemudian dibungkus dan dikembalikan dalam wujud ‘HighlightViewHolder’. Setelah itu, fungsi ‘getItemCount’ digunakan untuk mengembalikan total jumlah data melalui pemanggilan ‘games.size’ agar sistem RecyclerView mengetahui seberapa banyak batasan elemen yang perlu dimuat ke layar. Selanjutnya, fungsi ‘onBindViewHolder’ dipanggil untuk menarik elemen data dari list berdasarkan indeks urutan ‘position’, lalu melemparkan objek data tersebut beserta aksi kliknya ke dalam metode ‘bind’ yang ada di ViewHolder.

Pada bagian bawah, program mendefinisikan inner class ‘HighlightViewHolder’ yang berfungsi sebagai penampung referensi komponen UI dari XML. Saat kelas ViewHolder ini dikonstruksi, program langsung mencari elemen gambar menggunakan ‘findViewById(R.id.image_highlight)’ dan memetakannya ke dalam variabel lokal

‘imageGame’. Proses akhir penempelan data terjadi di dalam fungsi ‘bind’. Program memuat aset gambar ke tampilan dengan menyematkan ID resource gambar dari objek ‘game’ ke komponen ‘imageGame’ melalui fungsi ‘setImageResource’. Sebagai penyempurna interaksi, program menautkan ‘setOnClickListener’ secara langsung pada ‘itemView’ (yang mencakup seluruh area permukaan luar dari item tersebut). Apabila item sorotan itu diklik, program akan memicu fungsi lambda ‘onClick’ dan mengirimkan spesifikasi objek ‘game’ ke komponen induk (Activity atau Fragment) tempat adapter ini beroperasi, yang nantinya digunakan untuk berpindah ke halaman detail.

Package ui => home - HomeFragment.kt

File HomeFragment.kt ini bertindak sebagai halaman utama aplikasi yang diimplementasikan menggunakan struktur Fragment. Di bagian awal kelas, program mengelola antarmuka melalui sistem View Binding dengan mendeklarasikan variabel ‘_binding’ (bertipe nullable) dan ‘binding’. Pada siklus hidup ‘onCreateView’, layout XML dirangkai (di-inflate) ke dalam memori. Setelah tampilan dasar berhasil dibuat, siklus ‘onViewCreated’ langsung mengambil alih untuk mengeksekusi tiga fungsi utama yang menyiapkan interaksi elemen UI: ‘setupMenu’, ‘setupHighlightList’, dan ‘setupGameList’.

Pada fungsi ‘setupMenu’, program memasangkan listener pada komponen ‘toolbarHome’ melalui ‘setOnMenuItemClickListener’. Menggunakan blok percabangan ‘when’, program memantau ID menu yang ditekan; jika pengguna mengetuk ikon dengan ID ‘R.id.menu_settings’, maka perintah ‘findNavController().navigate’ akan memindahkan layar dari rute saat ini menuju ‘settingsFragment’. Selain itu, teks judul pada toolbar juga diinisialisasi di sini menggunakan nilai dari string resource.

Masuk ke fungsi ‘setupHighlightList’, bagian ini bertugas menangani tampilan daftar game sorotan pada komponen ‘rvHighlight’. Komponen RecyclerView tersebut diatur menggunakan ‘LinearLayoutManager’ dengan orientasi ‘HORIZONTAL’ agar daftarnya bisa digulir ke samping. Setelah itu, program menyematkan ‘HighlightAdapter’ yang sudah dibuat sebelumnya, memuat himpunan data dari ‘GameData.gameList’, dan mendefinisikan aksi klik. Ketika item diklik, fungsi lambda akan membungkus atribut ‘id’ dari game yang dipilih ke dalam sebuah objek ‘Bundle’, lalu mengirimkan data tersebut ke ‘detailFragment’ lewat navigasi.

Proses yang sangat mirip juga diterapkan pada fungsi ‘setupGameList’ untuk komponen ‘rvGames’, namun bedanya daftar ini dibiarkan menggunakan layout vertikal secara default.

Terakhir, terdapat dua penanganan siklus hidup krusial di bagian bawah. Fungsi ‘onDestroyView’ di-override untuk mencegah terjadinya kebocoran memori (memory leak) dengan mengosongkan variabel ‘_binding’ menjadi null persis sesaat setelah Fragment dihancurkan dari layar. Sementara itu, siklus ‘onResume’ ditambahkan secara khusus untuk melakukan penyegaran (refresh) pada teks judul toolbar.

Package settings - SettingsFragment.kt

File SettingsFragment.kt pada package ‘com.example.modul3xml.ui.settings’ merupakan halaman antarmuka pengaturan yang diimplementasikan menggunakan struktur Fragment dan sistem View Binding. Pada bagian awal kelas, dideklarasikan variabel ‘_binding’ yang nullable dan ‘binding’ sebagai getter non-null. Pemisahan ini wajib dilakukan pada Fragment untuk mencegah kebocoran memori. Pada siklus hidup ‘onCreateView’, tata letak XML di-inflate atau dirangkai ke dalam memori melalui ‘FragmentSettingsBinding.inflate’, kemudian elemen ‘root’ dikembalikan agar layarnya bisa dirender.

Setelah tampilan dasar siap, siklus ‘onViewCreated’ mengambil alih untuk mengeksekusi dua fungsi utama: ‘setupToolbar’ dan ‘setupLanguageButtons’. Di dalam fungsi ‘setupToolbar’, teks judul pada toolbar diatur, dan listener dipasang pada tombol panah kembali. Apabila tombol tersebut ditekan, perintah ‘findNavController().popBackStack()’ akan dijalankan untuk menghapus Fragment pengaturan ini dari riwayat navigasi dan mengembalikan layar ke halaman sebelumnya.

Pada fungsi ‘setupLanguageButtons’, program mengatur aksi pada dua tombol pilihan bahasa. Ketika ‘buttonIndonesia’ atau ‘buttonEnglish’ diklik, program akan mengeksekusi kelas utilitas eksternal ‘LanguageManager.setLocale’ dengan melemparkan referensi Activity saat ini melalui ‘requireActivity()’ beserta kode bahasanya (“id” atau “en”). Segera setelah konfigurasi bahasa sistem diubah oleh manager tersebut, fungsi pembantu ‘refreshTexts()’ langsung dipanggil.

Fungsi ‘refreshTexts’ ini secara spesifik bertugas untuk menimpa ulang teks pada ‘toolbarSettings’, ‘buttonIndonesia’, dan ‘buttonEnglish’ dengan cara memanggil kembali fungsi ‘getString’. Sebagai penutup, pada siklus ‘onDestroyView’, variabel ‘_binding’ diset

kembali menjadi null untuk memastikan semua referensi visual di dalam kelas benar-benar terhapus dari memori saat Fragment dihancurkan.

Package util - LanguageManager.kt

File LanguageManager.kt pada package ‘com.example.modul3xml.util’ ini dikonstruksi sebagai kelas utilitas (utility class) untuk mengelola konfigurasi bahasa aplikasi secara terpusat. Di baris awal, didefinisikan dua konstanta private yaitu ‘PREF_NAME’ dan ‘KEY_LANG’ yang bertindak sebagai nama file dan kunci pengenal untuk penyimpanan data lokal.

Logika penyimpanan status bahasa ditangani oleh dua fungsi private di bagian bawah, yakni ‘getSavedLanguage’ dan ‘saveLanguage’. Fungsi ini mengandalkan ‘SharedPreferences’ dengan hak akses ‘Context.MODE_PRIVATE’ untuk menyimpan kode bahasa yang diplih pengguna (misalnya “id” atau “en”) secara permanen ke dalam penyimpanan internal perangkat.

Untuk urusan eksekusi pergantian bahasa, terdapat fungsi ‘applySavedLocale’ dan ‘setLocale’. Fungsi ini memanggil ‘AppCompatDelegate.setApplicationLocales’ yang dipadukan dengan pemformatan dari ‘LocaleListCompat.forLanguageTags’. Melalui metode ini, sistem Android akan menangani perubahan referensi resource bahasa secara otomatis dan jauh lebih mulus. Fungsi ‘applySavedLocale’ diletakkan untuk dipanggil saat aplikasi baru pertama kali dimuat (booting), sedangkan ‘setLocale’ diikat pada aksi tombol di halaman pengaturan sekaligus menyimpannya ke SharedPreferences.

Sebagai pelengkap, dibuat juga sebuah fungsi ‘isIndonesian’ yang mengembalikan nilai boolean. Fungsi ini bertugas membaca langsung kode bahasa yang sedang digunakan oleh sistem UI saat ini lewat jalur ‘context.resources.configuration.locales[0].language’.

MainActivity.kt

File MainActivity.kt merupakan kelas utama yang mewarisi kelas ‘AppCompatActivity’ dan bertindak sebagai titik masuk (entry point) aplikasi. Pada bagian awal kelas, dideklarasikan variabel ‘binding’ bertipe ‘ActivityMainBinding’ menggunakan keyword ‘lateinit var’. Deklarasi ini menunda inisialisasi variabel hingga fungsi ‘onCreate’ dieksekusi untuk mengoptimalkan alokasi memori di awal. Variabel ini digunakan untuk

mengimplementasikan View Binding, yang memetakan seluruh elemen visual dari file tata letak XML ke dalam kode Kotlin.

Pada siklus hidup ‘onCreate’, eksekusi pertama yang dilakukan adalah pemanggilan fungsi ‘LanguageManager.applySavedLocale(this)’. Posisi pemanggilan ini sengaja diletakkan sebelum ‘super.onCreate(savedInstanceState)’. Penempatan ini bertujuan untuk menerapkan preferensi konfigurasi bahasa yang tersimpan di memori perangkat terlebih dahulu sebelum sistem Android mulai membangun dan merender komponen antarmuka.

Setelah konfigurasi bahasa diterapkan ke dalam context, program melakukan inflate atau pemuatan tata letak XML ke dalam memori menggunakan fungsi ‘ActivityMainBinding.inflate(layoutInflater)’, lalu menyimpannya ke variabel ‘binding’. Terakhir, perintah ‘setContentView(binding.root)’ dijalankan untuk menampilkan akar elemen visual dari layout tersebut ke layar perangkat. Activity ini bertindak sebagai wadah kosong utama, di mana alur perpindahan antarmuka di dalamnya akan dikendalikan sepenuhnya oleh sistem navigasi Fragment.

Package res => layout - activity_main.xml

File XML ini merupakan tata letak dasar untuk antarmuka utama aplikasi. Komponen akar yang digunakan adalah ‘FrameLayout’. Atribut ‘layout_width’ dan ‘layout_height’ diatur dengan nilai ‘match_parent’ agar ukurannya mengambil seluruh ruang layar secara penuh. Atribut ‘android:background’ ditambahkan untuk mengatur warna latar belakang layar menggunakan referensi ‘@color/background_black’.

Di dalam struktur layout tersebut, diletakkan sebuah komponen ‘androidx.fragment.app.FragmentContainerView’. Komponen ini berfungsi sebagai wadah kosong tempat halaman-halaman Fragment akan dimuat dan diganti secara dinamis. Atribut ‘android:name’ diisi dengan ‘androidx.navigation.fragment.NavHostFragment’, yang mengubah fungsi komponen ini menjadi pengontrol utama untuk sistem navigasi di dalam aplikasi.

Agar alur navigasi terhubung dengan interaksi perangkat, atribut ‘app:defaultNavHost=“true”’ ditambahkan supaya wadah ini bisa menangani langsung aksi dari tombol kembali (back button) bawaan Android. Terakhir, atribut ‘app:navGraph’ digunakan untuk menghubungkan wadah ini dengan berkas aturan navigasi

‘@navigation/nav_graph’. Dari sinilah sistem membaca halaman Fragment mana yang harus dimuat pertama kali dan bagaimana alur perpindahan antarhalamannya.

Package res => layout - fragment_detail.xml

File XML ini merupakan antarmuka untuk halaman detail game. Struktur paling luar menggunakan komponen ‘LinearLayout’ dengan orientasi ‘vertical’ agar elemen-elemen di dalamnya tersusun secara urut dari atas ke bawah. Atribut ‘android:fitsSystemWindows= “true”’ ditambahkan agar tampilan konten tidak tertabrak atau tertutup oleh bar status (status bar) bawaan sistem Android. Di urutan paling atas, diletakkan komponen ‘MaterialToolbar’ dengan ID ‘toolbar_detail’ yang berfungsi sebagai bar navigasi atas. Toolbar ini sudah langsung dilengkapi dengan ikon panah kembali melalui atribut ‘app:navigationIcon’, serta warna teks dan ikonnya diatur menjadi putih agar terlihat jelas di atas latar warna hitam.

Di bawah toolbar, disematkan komponen ‘ScrollView’ supaya area konten bisa digulir ke bawah apabila isi teksnya panjang dan melebihi tinggi layar. Atribut ‘android:fillViewport= “true”’ ditambahkan agar area gulir tersebut bisa merentang memakan seluruh sisa layar secara penuh. Di dalam ‘ScrollView’, dibuat sebuah ‘LinearLayout’ vertikal lagi sebagai wadah pembungkus susunan elemen konten yang diberi jarak tepi menggunakan padding sebesar ‘16dp’. Komponen konten pertama adalah ‘ImageView’ dengan ID ‘image_detail’ untuk menampilkan gambar sampul. Agar tidak terlihat gepeng, gambar diatur agar terpotong proporsional memenuhi area komponen menggunakan ‘android:scaleType= “centerCrop”’. Kemudian, sudut gambar dibuat melengkung dengan menggabungkan atribut ‘android:background= “@drawable/bg_image_rounded”‘ dan ‘android:clipToOutline= “true”’.

Tepat di bawah gambar, terdapat sebuah ‘LinearLayout’ berorientasi ‘horizontal’ untuk meletakkan teks judul dan tahun rilis supaya posisinya bisa berdampingan. Pada komponen ‘TextView’ untuk judul (‘text_detail_title’), atribut ‘android:layout_width= “0dp”’ dan ‘android:layout_weight= “1”’ dipasangkan supaya elemen judul ini menyita semua sisa ruang horizontal yang kosong. Pengaturan ini otomatis mendorong komponen ‘TextView’ tahun (‘text_detail_year’) menjadi sejajar dan menempel rapat di ujung kanan layar. Pada bagian paling bawah, disusun dua komponen ‘TextView’ tambahan. ‘TextView’ yang pertama berisi teks statis “Plot” yang dicetak tebal sebagai subjudul, sedangkan yang kedua memiliki ID

‘text_detail_plot’ dan dibiarkan tanpa teks awal karena komponen ini akan digunakan sebagai wadah untuk menampung deskripsi alur cerita yang datanya dilempar dari kode Kotlin.

Package res => layout - fragment_home.xml

File XML ini merupakan rancangan antarmuka untuk halaman utama aplikasi. Struktur paling luar menggunakan komponen ‘LinearLayout’ dengan orientasi ‘vertical’ agar semua elemen di dalamnya tersusun rapi dari atas ke bawah. Atribut ‘android:fitsSystemWindows= “true”’ digunakan agar tata letak tidak tertutup oleh bar status dari sistem Android, sedangkan latar belakangnya diatur menjadi hitam melalui ‘@color/background_black’. Di bagian paling atas, terdapat ‘MaterialToolbar’ dengan ID ‘toolbar_home’ yang berfungsi sebagai bar navigasi. Toolbar ini dilengkapi dengan ikon menu yang dipanggil melalui atribut ‘app:menu=“@menu/menu_home”’ dan warna teks judulnya diatur menjadi putih.

Di bawah toolbar, program menggunakan ‘androidx.core.widget.NestedScrollView’ agar seluruh konten di bawahnya bisa digulir secara vertikal. Atribut ‘android:fillViewport=“true”’ memastikan area gulir mengambil seluruh sisa tinggi layar secara penuh, sementara ‘android:overScrollMode=“never”’ digunakan untuk menghilangkan efek bayangan saat pengguna menggulir layar hingga ke ujung atas atau bawah. Di dalam komponen ini, dibuat sebuah ‘LinearLayout’ vertikal lagi sebagai wadah pembungkus untuk menyusun elemen-elemen daftar.

Elemen konten yang pertama adalah sebuah ‘TextView’ dengan ID ‘text_highlight’ yang menampilkan subjudul dengan cetak tebal berwarna putih. Tepat di bawahnya, disematkan komponen ‘RecyclerView’ pertama dengan ID ‘rv_highlight’ yang diberi tinggi tetap ‘120dp’ untuk menampilkan daftar game sorotan (highlight) secara horizontal. Penggunaan atribut ‘android:clipToPadding=“false”’ pada komponen ini berfungsi agar item di dalamnya tetap bisa digulir melewati batas margin kiri dan kanan tanpa terpotong.

Terakhir, terdapat ‘RecyclerView’ kedua dengan ID ‘rv_games’ untuk menampilkan daftar game utama. Pada kedua daftar ‘RecyclerView’ ini, atribut ‘android:nestedScrollingEnabled=“false”’ sengaja ditambahkan. Pengaturan ini sangat penting karena daftar ini berada di dalam sebuah ‘NestedScrollView’; dengan mematikan fungsi gulir bawaan dari ‘RecyclerView’, sistem akan menyerahkan kendali gulir sepenuhnya

kepada komponen induknya, sehingga pergerakan layar saat di-scroll ke bawah akan terasa lebih mulus dan tidak saling bertabrakan.

Package res => layout - fragment_settings.xml

File XML ini merupakan rancangan antarmuka untuk halaman pengaturan. Struktur paling luar menggunakan komponen 'LinearLayout' dengan orientasi 'vertical' agar semua elemen di dalamnya tersusun rapi dari atas ke bawah. Atribut 'android:fitsSystemWindows="true"' ditambahkan agar tata letak tampilan tidak tertabrak oleh bar status dari sistem Android. Latar belakang layar juga langsung diatur menjadi hitam melalui atribut 'android:background="@color/background_black"'.

Di urutan paling atas, diletakkan komponen 'MaterialToolbar' dengan ID 'toolbar_settings' yang berfungsi sebagai bar navigasi atas. Tinggi komponen ini disesuaikan dengan standar bawaan sistem menggunakan '?attr/actionBarSize'. Toolbar tersebut sudah dilengkapi dengan ikon panah kembali yang dipanggil lewat atribut 'app:navigationIcon', sementara warna ikon dan teks judulnya diatur menjadi putih supaya terlihat kontras di atas latar gelap.

Tepat di bawah toolbar, terdapat sebuah 'LinearLayout' kedua yang juga berorientasi 'vertical' untuk membungkus opsi pilihan bahasa. Agar letak tombol-tombol nantinya berada pas di tengah layar, atribut 'android:gravity="center"' disematkan pada wadah ini. Selain itu, diberikan juga ruang kosong di sisi kiri dan kanan melalui 'android:paddingHorizontal="48dp"' supaya ukuran tombol tidak terlalu melebar dan terlihat lebih proporsional.

Di dalam wadah penengah tersebut, disusun dua buah tombol menggunakan komponen 'MaterialButton'. Tombol pertama memiliki ID 'button_indonesia' dan tombol kedua memiliki ID 'button_english', di mana lebar keduanya diatur agar membentang memenuhi wadahnya menggunakan 'match_parent'. Kedua tombol ini menggunakan warna latar biru yang dipanggil melalui atribut 'app:backgroundTint="@color/button_blue"'. Sebagai pemisah, disisipkan komponen 'Space' dengan tinggi '16dp' di antara kedua tombol tersebut agar tampilannya rapi dan tidak saling menempel.

Package res - layout - item_game.xml

File XML ini berfungsi sebagai cetakan desain (blueprint) untuk masing-masing baris item game pada daftar utama (RecyclerView). Komponen terluarnya menggunakan 'MaterialCardView' untuk memberikan bentuk kartu dengan sudut melengkung '16dp' dan efek bayangan tipis ('app:cardElevation= "2dp"'). Latar belakang kartu ini diatur menjadi abu-abu gelap melalui referensi '@color/card_dark', dan garis tepinya dihilangkan dengan 'app:strokeWidth= "0dp"'.

Di dalam kartu tersebut, terdapat 'LinearLayout' utama dengan orientasi 'horizontal' yang membagi area menjadi sisi kiri dan sisi kanan. Pada sisi kiri, disematkan komponen 'ImageView' dengan ID 'image_game' untuk menampilkan gambar sampul. Agar sudut gambar ikut melengkung dan tidak menabrak batas kartu, digunakan atribut 'android:background= "@drawable/bg_image_rounded"' yang dipadukan dengan 'android:clipToOutline= "true"'. Sementara itu, potongan gambarnya diatur proporsional menggunakan 'android:scaleType= "centerCrop"'.

Beralih ke sisi kanan, terdapat 'LinearLayout' vertikal yang menampung rincian teks dan tombol interaksi. Di bagian atasnya, ada susunan horizontal yang menampilkan komponen 'TextView' untuk judul game ('text_title') dan tahun rilis ('text_year'). Teks judul dirancang agar menyita ruang kosong ('android:layout_weight= "1"') sehingga teks tahun akan merapat ke kanan. Tepat di bawahnya, ada 'TextView' untuk plot cerita ('text_plot'). Agar tampilan kartu tetap rapi, teks judul dibatasi hanya satu baris ('android:maxLines= "1"') dan plot cerita dibatasi maksimal dua baris ('android:maxLines= "2"'). Jika teksnya melebihi batas, akan otomatis terpotong dengan tanda elipsis (...) berkat atribut 'android:ellipsize= "end"'.

Pada bagian bawah kolom sisi kanan ini, diletakkan komponen 'Space' dengan bobot ('layout_weight= "1"') yang berfungsi sebagai pendorong sisa elemen ke bawah. Berkat dorongan ini, dua buah 'MaterialButton' yang terbungkus di dalam wadah horizontal di bawahnya (tombol "Browser" dan "Detail") akan terdorong rapi merapat ke sudut kanan bawah, menjadikannya sejajar dengan ujung bawah gambar sampul. Kedua tombol tersebut diatur agar ukurannya lebih ramping dengan tinggi '36dp' dan mewarisi warna latar belakang kustom '@color/button_blue'.

Package res => layout - item_highlight_game.xml

File XML ini dirancang sebagai cetakan tampilan untuk masing-masing item game pada daftar sorotan (highlight) yang menggulir ke samping. Komponen dasarnya menggunakan 'MaterialCardView' dengan ukuran tetap, yaitu lebar '200dp' dan tinggi '120dp'. Agar antar-item tidak saling menempel rapat saat ditampilkan berjejer di dalam RecyclerView, ditambahkan jarak luar di sisi kanan sebesar '12dp' melalui atribut 'android:layout_marginEnd'. Kartu ini juga didesain dengan sudut melengkung sebesar '12dp', diberi efek bayangan tipis dengan elevasi '2dp', serta garis tepinya dihilangkan menggunakan 'app:strokeWidth= " 0dp"'.

Di dalam kartu tersebut, hanya disematkan satu buah komponen, yaitu 'ImageView' dengan ID 'image_highlight'. Komponen ini bertugas untuk memuat gambar sampul game. Atribut lebar dan tingginya diatur menjadi 'match_parent' agar gambar bisa membentang penuh menutupi seluruh area dalam kartu. Agar tampilan gambar tetap proporsional dan tidak terlihat merenggang atau gepeng saat dipaksa menyesuaikan dengan dimensi kartu, digunakan atribut 'android:scaleType=" centerCrop"'.

Package res => menu - menu_home.xml

File XML ini difungsikan sebagai kerangka untuk membuat menu navigasi, yang biasanya disematkan pada komponen toolbar di bagian atas aplikasi. Struktur utamanya dibungkus menggunakan elemen akar '<menu>'. Di dalamnya, diletakkan sebuah elemen '<item>' yang merepresentasikan satu tombol aksi pada menu tersebut.

Pada elemen '<item>' ini, disematkan atribut 'android:id="@+id/menu_settings"' sebagai identitas unik. ID ini sangat penting agar tombol tersebut bisa dikenali dan ditangani aksi kliknya nanti di dalam kode Kotlin (seperti pada halaman utama sebelumnya). Atribut 'android:icon' digunakan untuk memanggil aset gambar dari folder drawable yang akan ditampilkan sebagai ikon visual tombol, sedangkan 'android:title' memuat teks judul dari string resource yang berfungsi sebagai keterangan bantuan teks jika menu diklik-tahan atau jika ikon gagal dimuat.

Terakhir, atribut 'app:showAsAction="ifRoom"' ditambahkan sebagai aturan tata letak menu. Pengaturan ini menginstruksikan sistem agar memprioritaskan item menu ini untuk tampil langsung sebagai ikon di area toolbar selama masih ada ruang kosong. Jika ruang pada

toolbar sudah penuh, item ini otomatis akan disembunyikan dan dipindahkan ke dalam menu tumpuk (ikon titik tiga atau overflow menu).

Package res => navigation - nav_graph.xml

File XML ini berfungsi sebagai peta navigasi (navigation graph) yang mengatur alur perpindahan antarhalaman di dalam aplikasi. Struktur utamanya dibungkus oleh elemen <navigation>. Pada elemen akar ini, terdapat atribut app:startDestination="@id/homeFragment" yang berfungsi menetapkan halaman mana yang akan langsung dimuat pertama kali saat aplikasi dijalankan.

Di dalam peta ini, kita mendaftarkan tiga buah halaman menggunakan elemen <fragment>, yaitu homeFragment, detailFragment, dan settingsFragment. Pada bagian homeFragment, ditambahkan dua buah elemen <action> yang mendefinisikan rute atau jalur spesifik untuk berpindah layar. Jalur pertama menggunakan ID action_homeFragment_to_detailFragment untuk menghubungkan halaman utama ke halaman detail, sedangkan jalur kedua menggunakan ID action_homeFragment_to_settingsFragment untuk mengarah ke halaman pengaturan.

Selain mengatur halaman tujuan (app:destination), setiap elemen <action> juga memuat empat atribut animasi (enterAnim, exitAnim, popEnterAnim, popExitAnim). Atribut-atribut ini memanggil referensi file dari folder animasi untuk memberikan efek visual bergeser (slide) saat pengguna berpindah halaman atau saat menekan tombol kembali. Sementara itu, elemen detailFragment dan settingsFragment di bagian bawah hanya didaftarkan referensi kelasnya saja tanpa memuat <action> tambahan. Hal ini karena kedua halaman tersebut hanya menjadi titik tujuan akhir, dan proses navigasi kembalinya cukup ditangani oleh fungsi bawaan dari sistem back stack Android.

SOAL 2

Mengapa RecyclerView masih digunakan, padahal RecyclerView memiliki kode yang panjang dan bersifat boiler-plate, dibandingkan LazyColumn dengan kode yang lebih singkat?

A. Pembahaan

RecyclerView masih digunakan karena mayoritas proyek Android yang ada saat ini dibangun menggunakan XML dan View System, bukan Jetpack Compose. Migrasi dari RecyclerView ke LazyColumn akan memakan waktu dan effort yang tidak sedikit karena membutuhkan perubahan arsitektur yang cukup besar, sehingga banyak tim atau perusahaan memilih untuk tetap menggunakan RecyclerView demi efisiensi waktu dan stabilitas proyek.

Selain itu, RecyclerView sudah sangat matang dan telah teruji di berbagai kondisi produksi selama bertahun-tahun. Dokumentasinya lengkap, komunitas penggunanya luas, dan sebagian besar developer Android sudah familiar dengan cara kerjanya. Meskipun kodenya lebih Panjang dan terasa boilerplate, bagi developer yang sudah terbiasa, hal itu bukan masalah berarti.

LazyColumn sendiri hanya bisa digunakan dalam ekosistem Jetpack Compose. Jadi selama sebuah proyek masih menggunakan pendekatan XML tradisional, RecyclerView tetap menjadi pilihan utama karena memang tidak ada alternatif lain di dalam sistem tersebut.

SOAL 3

Pada desain arsitektur kotlin, terdapat beberapa arsitektur dengan salah satunya adalah CLEAN ARCHITECTURE, menurut **pendapat kalian**, mengapa kita harus menerapkan arsitektur ini ketika membuat sebuah project?

A. Pembahasan

Menurut saya, Clean Architecture perlu diterapkan terutama ketika kita membangun proyek yang tidak sederhana dan kemungkinan besar akan terus berkembang. Alasan utamanya adalah pemisahan tanggung jawab antar lapisan kode. Dengan adanya pemisahan tersebut, setiap bagian dari aplikasi punya tugasnya masing-masing dan tidak saling bergantung secara langsung, sehingga ketika ada perubahan di satu bagian, bagian lainnya tidak ikut terdampak.

Kode yang tidak terstruktur memang terasa lebih cepat di awal pengembangan, tapi begitu proyeknya mulai besar, kode tersebut jadi susah dibaca dan susah diubah. Clean Architecture membantu menghindari kondisi seperti itu karena struktur foldernya sudah mencerminkan alur logika dari aplikasi itu sendiri.

Selain itu, Clean Architecture juga memudahkan proses pengujian. Karena logika bisnis tidak tercampur dengan logika tampilan atau sumber data, kita bisa menguji bagian tertentu secara terpisah tanpa harus menjalankan keseluruhan aplikasi.

ADDITIONAL

Penerapan dan pembahasan clean code

A. Pembahasan

COMPOSE

Saya susun folder berdasarkan fungsinya, jadi ada folder terpisah untuk data, model, navigation, dan screen. Ini membuat struktur proyek lebih mudah dipahami saat pertama kali membuka. Setiap screen saya buat dalam file tersendiri dengan nama yang jelas seperti HomeScreen.kt, DetailScreen.kt, SettingsScreen.kt sehingga file tidak jadi terlalu besar dan rumit. Saya juga memisahkan komponen UI menjadi bagian-bagian yang lebih kecil. Di HomeScreen, ada komponen 'GameListItem' yang terpisah, jadi saat menggambar satu item game tidak perlu ikut-ikutan dengan logika seluruh list. Model Game saya buat sebagai data class dengan field yang jelas, sehingga struktur datanya langsung terlihat tanpa perlu baca dokumentasi panjang.

XML

Penerapan clean code terlihat dari cara saya membuat fragment. Di HomeFragment, saya membagi kerjanya ke dalam beberapa method seperti 'setupMenu()', 'setupHighlightList()', dan 'setupGameList()'. Setiap method punya fokus tersendiri sehingga kalau ada bug atau perlu diubah, tinggal lihat method yang relevan tanpa perlu cari-cari di seluruh file.

Kemudian, saya menyadari bahwa pengecekan bahasa dipakai di berbagai tempat, jadi saya buat LanguageManager untuk menangani semuanya. Dengan cara ini, kalau pengecekan bahasa perlu diubah, tinggal edit satu tempat saja. Dalam layout fragment_home.xml, saya juga sengaja pakai resource untuk teks dan warna seperti '@string/title_highlight', '@color/white' daripada tulis nilai langsung. Ini membuat kode lebih fleksibel kalau ada perubahan tampilan. Di GameData, saya tambah method 'findGameById()' sehingga proses pencarian data jadi lebih standar dan bisa dipakai di mana saja.

TAUTAN GIT

Berikut adalah tautan untuk repositori GitHub yang telah dibuat

https://github.com/Fiezz65/MODUL-MOBILE/tree/master/MODUL_3